BÀI TẬP + LỜI GIẢI CHI TIẾT · SEMANTIC WEB

Toàn bộ bài tập tuần (Part 1–3, tableau OWL2, bài về nhà 29.08 · 5.9 · 12.9, 25 truy vấn SPARQL) và 15 câu của 3 đề cũ. Đi kèm sổ tay TAI-LIEU-IN.pdf: nhận dạng câu bằng sổ tay → lật tới cùng mã [Dx] ở đây → chép theo bài giống nhất.

Mục lục theo dạng

D1 · Câu văn → Turtle + vẽ đồ thị RDF **3**

D2 · Chuyển đổi đồ thị ↔ Turtle ↔ N-Triples ↔ RDF/XML **13**

D3 · RDFS: schema (lớp, thuộc tính, domain, range) .. **16**

D4 · Suy diễn RDFS: luật + cơ sở, true / unknown **19**

D5 · Thiết kế ontology OWL + biểu thức DL **22**

D6 · Suy diễn OWL / reasoner **32**

D12 · Tableau: NNF, 7 luật, consistency / subsumption **36**

D7 · Datalog: cây chứng minh + điểm cố định **40**

D8 · Viết truy vấn SPARQL (câu 2.5 điểm) **43**

D9 · Đại số SPARQL: dịch query → bảng từng bước .. **59**

D10 · SHACL: khuôn hình, nút được kiểm, vi phạm .. **62**

Bài → trang

Bài	Dạng	Trang
Part 1 (RDF, RDFS)		
Bài mẫu "liệt kê hết": Part 1 Ex1 (SOICT)	D1	3
Part 1 Ex2 — doc1/doc2/doc3 (bài về nhà 29.08)	D1	5
Part 1 Ex4 — Đồ thị Georges/Michael → Turtle	D2	13
Part 1 Ex6 — Turtle → đồ thị	D2	14
Part 1 Ex3 — URI1…URI6	D3	16
Part 1 Ex5 — Teacher/Professor/Assistant/Course (D3)	D3	17
Part 1 Ex7 — Thư viện (D3, đề mở)	D3	17
Bộ 3 — Thư viện (Part 1 Ex7) · dữ liệu library	D8	51
Part 2 (RDFS)		
Part 2 Ex1 — Mô tả ontology trong hình	D3	18
Part 2 Ex2 — Đồ thị gia đình (Person/Father/Ann/Carl)	D4	19
Part 3 (OWL)		
Lời giải Part 3 Ex3	D5	22
Lời giải Part 3 Ex2 (thuốc)	D5	25
Lời giải Part 3 Ex4 (phim)	D5	26
③ Bài tập tuần: Pizza (P3 Ex1)	D6	32
Bộ 1 — Ontology môn học (Part 3 Ex3) · dữ liệu ex3-lab-course	D8	43
Bộ 2 — Ontology phim (Part 3 Ex4) · dữ liệu ex4-film	D8	47
Tableau OWL2 (Exercises-OWL2)		
Ex1 — Consistency checking (Pingu)	D12	36
Ex2 — Class instantiation (Mary)	D12	37
Ex3 — Subsumption với lượng từ	D12	37
Đề 2023-3		

2023-3 · Câu 1 — Turtle + đồ thị RDF	D1	6
2023-3 · Câu 2 — RDFS + OWL + DL (phương tiện giao thông)	D5	27
2023-3 · Câu 3 — Datalog: cây chứng minh + điểm cố định	D7	41
2023-3 · Câu 4 — SPARQL + đại số (máy bay)	D8	54
2023-3 · Câu 5 — SHACL	D10	63
Đề 2022-1		
2022-1 · Câu 1 — Đồ thị RDF + Turtle (sách Design Patterns)	D1	8
2022-1 · Câu 2 — Suy diễn RDFS	D4	20
2022-1 · Câu 3 — RDFS hay OWL? (Molly là mèo, Spot thì không)	D6	33
2022-1 · Câu 4 — DL + RDFS/OWL (thú cưng)	D5	28
2022-1 · Câu 5 — SPARQL + đại số (mèo của Anna)	D8	55
Đề 6390		
6390 · Câu 1 — Turtle + đồ thị (C++)	D1	9
6390 · Câu 2 — RDFS + OWL (phần mềm C++)	D5	29
6390 · Câu 3 — Suy diễn OWL với allValuesFrom	D6	34
6390 · Câu 4 — SPARQL (ngôn ngữ lập trình) + đại số	D8	56
6390 · Câu 5 — SHACL	D10	64
Bài luyện thêm (D7, D9, D10, ví dụ slide)		
4. Ví dụ trên slide (nên tự làm lại trên giấy)	D12	38
Luyện tập 1 — Tổ tiên	D7	40
Luyện tập 2 — Chuyến bay khứ hồi	D7	40
Luyện tập 3 — Luật RDFS viết bằng Datalog	D7	41
Luyện tập 1 — FILTER bên trong OPTIONAL và FILTER bên ngoài	D9	59
Luyện tập 2 — Union + Join + Filter	D9	60
Luyện tập 3 — GROUP BY + COUNT/SUM	D9	60
Luyện tập 1 — Nhãn duy nhất	D10	62
Luyện tập 2 — Khoảng giá trị, chọn target theo thuộc tính	D10	62
Luyện tập 3 — sh:class và OWL allValuesFrom	D10	62
Luyện tập 4 — Giá trị liệt kê (sh:in)	D10	63

D1 Câu văn → Turtle + vẽ đồ thị RDF

Đề cho một đoạn văn → viết Turtle và vẽ đồ thị RDF, liệt kê HẾT các triple.

③ Bài mẫu + bài tập tuần

■ Bài mẫu "liệt kê hết": Part 1 Ex1 (SOICT)

Dạng: *D1* · ***Nguồn:*** *Exercises-en Part 1 Ex1 (ảnh bài tập 2) – mỗi câu ≥ 3 triple*

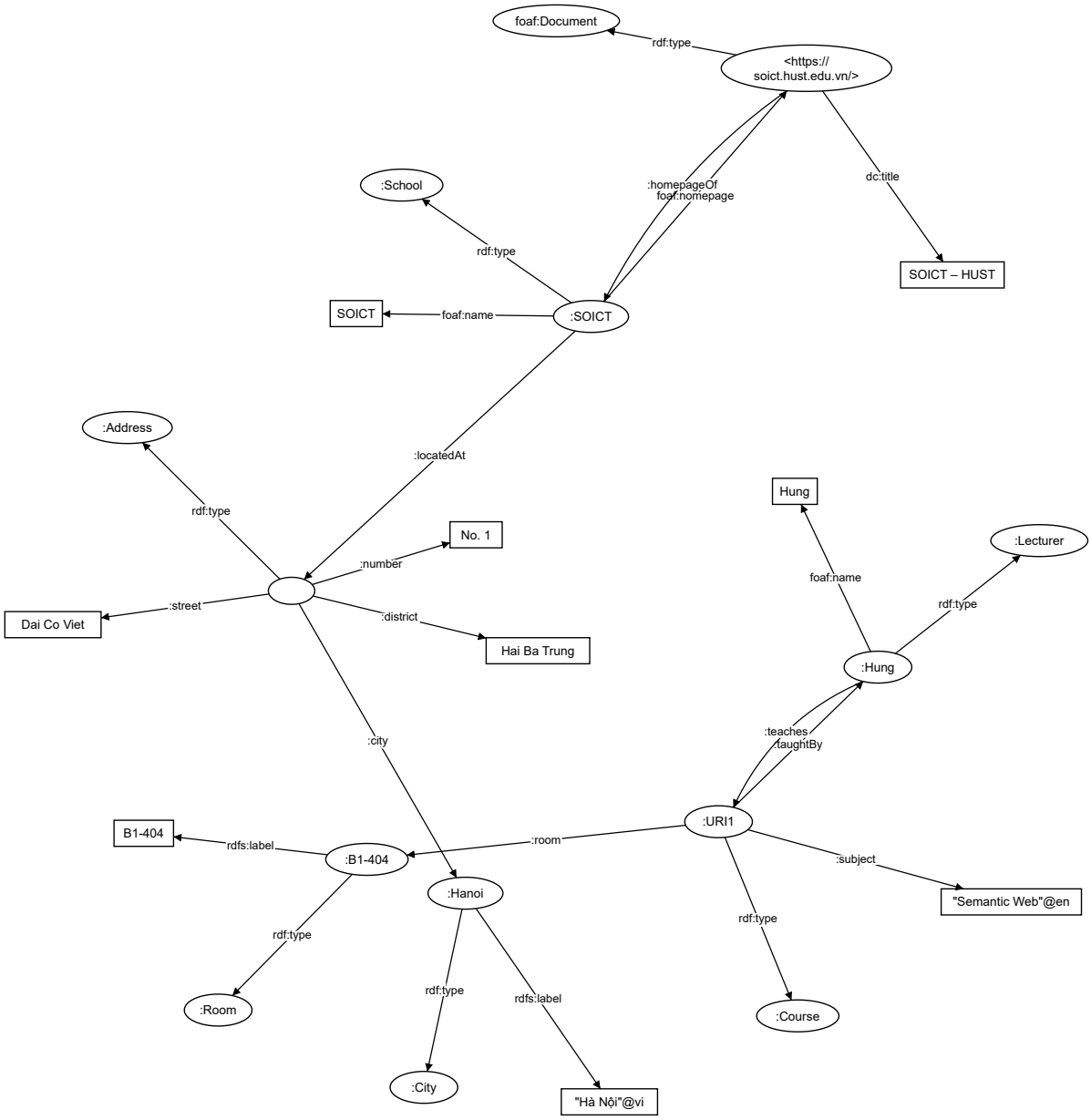
Đề: (1) SOICT is located at No. 1, Dai Co Viet, Hai Ba Trung, Hanoi. (2) Website https://soict.hust.edu.vn/ is the homepage of SOICT. (3) Resource URI1 is a course on Semantic Web, taught by Lecturer Hung in room B1-404. Viết dưới dạng ngôn ngữ tự nhiên (mỗi triple một câu) rồi viết RDF.

▶ Đáp án

Câu tự nhiên (mỗi dòng 1 triple):

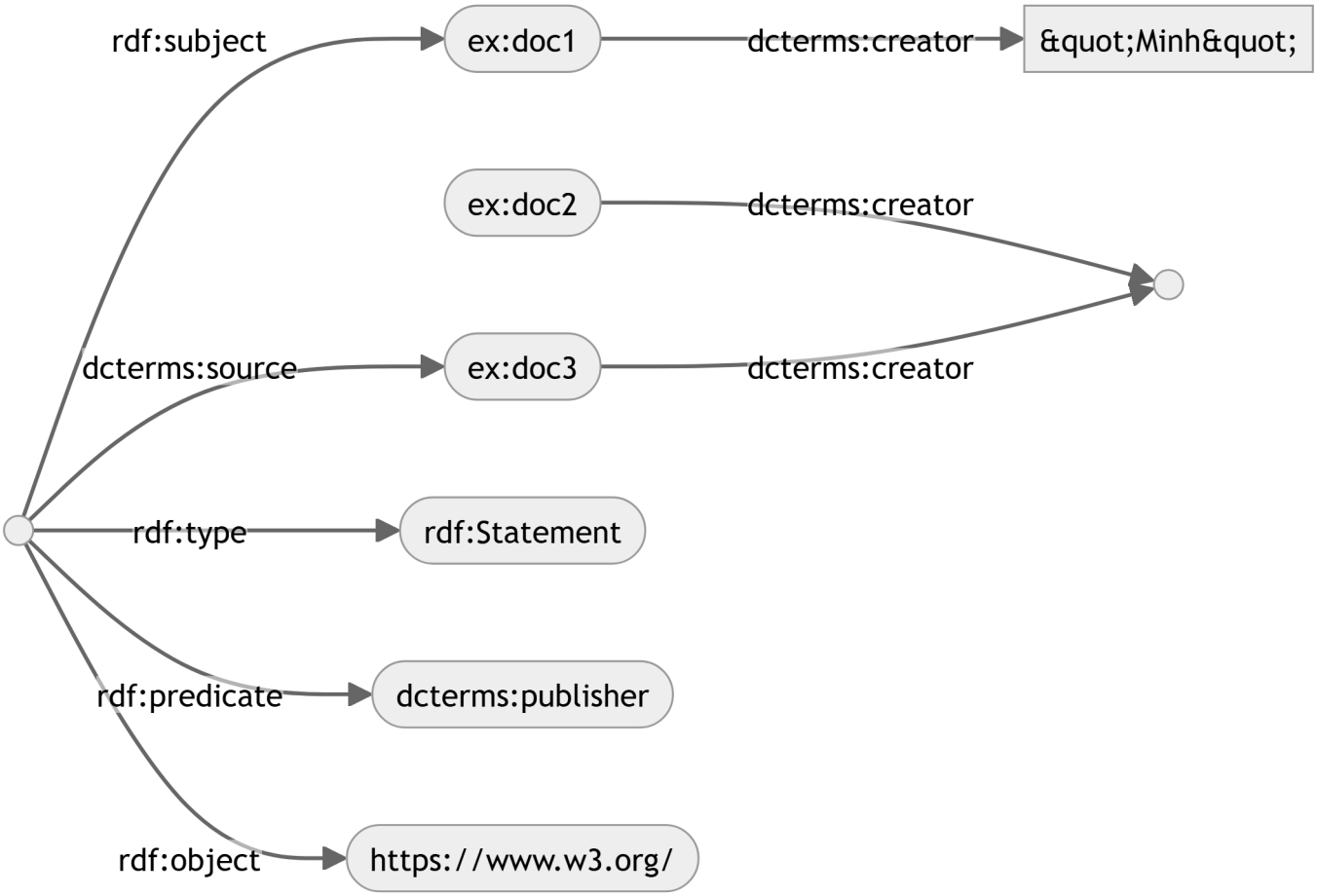
1. SOICT là một tổ chức (School). SOICT có tên "SOICT". SOICT nằm tại một địa chỉ (blank node). Địa chỉ đó có số nhà "No. 1". Địa chỉ đó thuộc đường "Dai Co Viet". Địa chỉ đó thuộc quận "Hai Ba Trung". Địa chỉ đó thuộc thành phố Hanoi. Hanoi là một City. Hanoi có nhãn "Hà Nội"@vi.
2. https://soict.hust.edu.vn/ là một Website (foaf:Document). SOICT có trang chủ https://soict.hust.edu.vn/. https://soict.hust.edu.vn/ là trang chủ của SOICT (quan hệ ngược). Trang có tiêu đề "SOICT – HUST".
3. URI1 là một Course. URI1 có chủ đề "Semantic Web". URI1 được dạy bởi Hung. Hung là một Lecturer. Hung có tên "Hung". URI1 được dạy ở phòng B1-404. B1-404 là một Room. B1-404 có nhãn "B1-404". Hung dạy URI1 (quan hệ ngược).

```
@prefix : <http://example.org/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix dc: <http://purl.org/dc/elements/1.1/> .
:SOICT a :School ; foaf:name "SOICT" ;
  :locatedAt [ a :Address ; :number "No. 1" ; :street "Dai Co Viet" ; :district "Hai Ba Trung" ;
:city :Hanoi ] ;
  foaf:homepage <https://soict.hust.edu.vn/> .
:Hanoi a :City ; rdfs:label "Hà Nội"@vi .
<https://soict.hust.edu.vn/> a foaf:Document ; :homepageOf :SOICT ; dc:title "SOICT – HUST" .
:URI1 a :Course ; :subject "Semantic Web"@en ; :taughtBy :Hung ; :room :B1-404 .
:Hung a :Lecturer ; foaf:name "Hung" ; :teaches :URI1 .
:B1-404 a :Room ; rdfs:label "B1-404" .
```



Đếm: câu 1 có 10 triple, câu 2 có 3–4 triple, câu 3 có 9 triple ⇒ mọi câu đều ≥ 3 triple. Khi đi thi: viết **đủ** kiểu, tên, quan hệ chính, quan hệ ngược.

■ Part 1 Ex2 — doc1/doc2/doc3 (bài về nhà 29.08)




```
@prefix rdf:     <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix dcterms: <http://purl.org/dc/terms/> .
@prefix ex:      <http://example.org/> .

ex:doc1 dcterms:creator "Minh" .
ex:doc2 dcterms:creator _:author .          # cùng MỘT tác giả không rõ danh tính -> cùng một
blank node
ex:doc3 dcterms:creator _:author .
_:stmt a rdf:Statement ;                    # doc3 NÓI RẰNG doc1 do W3C xuất bản -> reification
    rdf:subject   ex:doc1 ;
    rdf:predicate dcterms:publisher ;
    rdf:object    <https://www.w3.org/> ;
    dcterms:source ex:doc3 .
```

```
<?xml version="1.0" encoding="UTF-8"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:dcterms="http://purl.org/dc/terms/">
  <rdf:Description rdf:about="http://example.org/doc1">
    <dcterms:creator>Minh</dcterms:creator>
  </rdf:Description>
  <rdf:Description rdf:about="http://example.org/doc2">
    <dcterms:creator rdf:nodeID="author"/>
  </rdf:Description>
  <rdf:Description rdf:about="http://example.org/doc3">
    <dcterms:creator rdf:nodeID="author"/>
  </rdf:Description>
  <rdf:Statement rdf:nodeID="stmt">
    <rdf:subject rdf:resource="http://example.org/doc1"/>
    <rdf:predicate rdf:resource="http://purl.org/dc/terms/publisher"/>
    <rdf:object rdf:resource="https://www.w3.org/"/>
    <dcterms:source rdf:resource="http://example.org/doc3"/>
  </rdf:Statement>
</rdf:RDF>
```

Điểm ăn điểm: (1) blank node **dùng chung** cho doc2/doc3 (không phải hai blank node khác nhau — vì "cùng một tác giả"); (2) **reification** thay vì khẳng định trực tiếp `ex:doc1 dcterms:publisher <W3C>`; (3) biết `dcterms:creator` có range `dcterms:Agent` nên cách chặt chẽ hơn là `ex:doc1 dcterms:creator [ a dcterms:Agent ; rdfs:label "Minh" ]` (literal hợp lệ với `dc:` 1.1 cũ).

④ Đề thi cũ dạng này

■ 2023-3 · Câu 1 — Turtle + đồ thị RDF

> ***Dạng:*** *D1 · **Nguồn:** 2023-C1*

Tạo dữ liệu RDF theo cú pháp Turtle biểu diễn các thông tin, sau đó vẽ đồ thị RDF tương ứng:

Học phần The Semantic Web có danh mục sách tham khảo bao gồm: Foundations of Semantic Web Technologies, Learning SPARQL, Semantic Web for the Working Ontologist. Sinh viên Nguyễn Văn A đã đạt điểm 10 học phần này. Thầy giáo nhận xét Nguyễn Văn A là 1 sinh viên xuất sắc.

!Lưu ý: Tự tạo IRI cho các thành phần (khóa học, sách và sinh viên), chỉ viết không quá 20 bộ-3 cơ bản/20 cạnh đồ thị RDF.

◇ Gợi ý / Khuôn

Tách đề thành 3 mệnh đề và chọn cấu trúc theo **dấu hiệu** của từng mệnh đề:

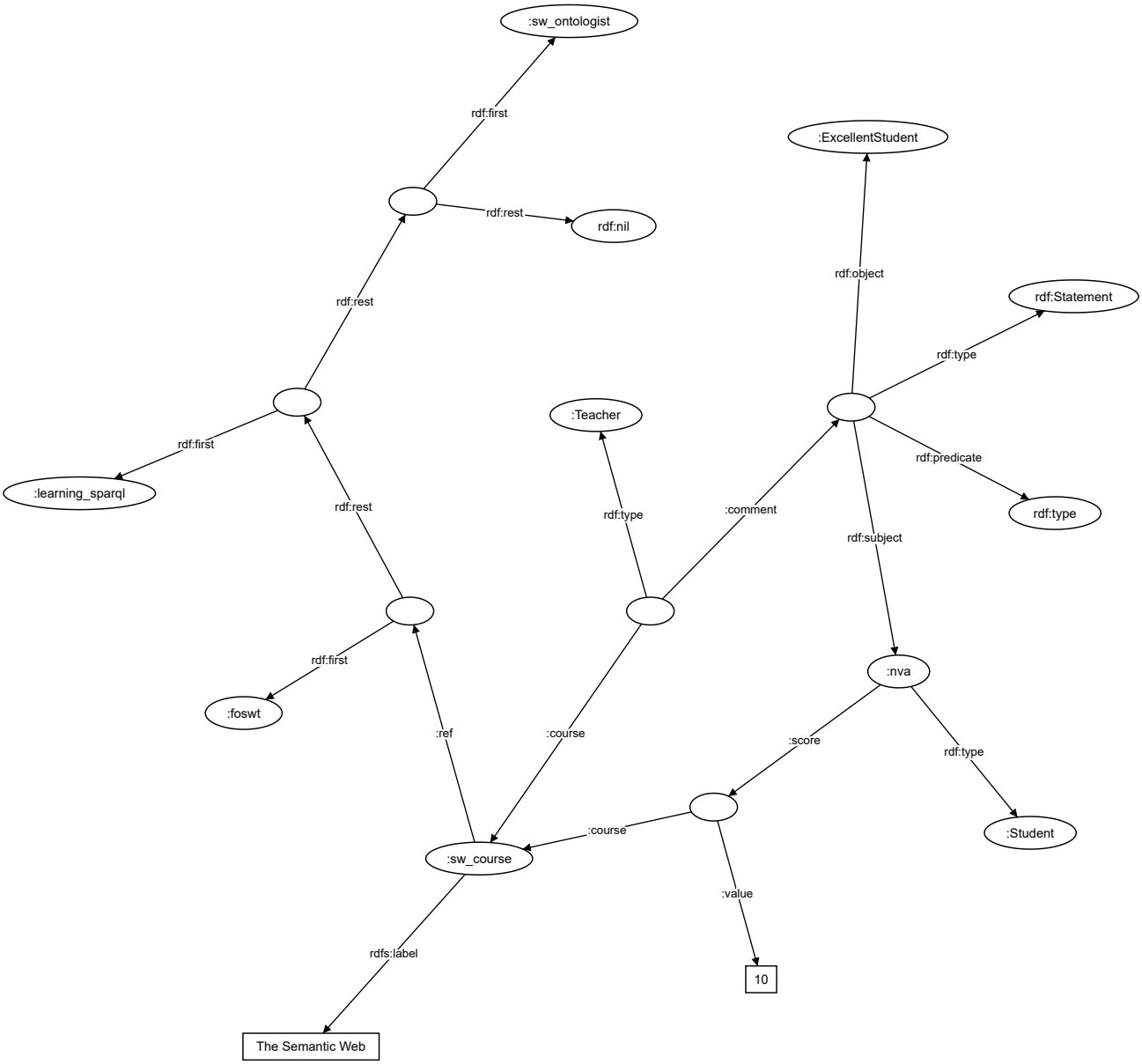
Cụm từ trong đề	Cấu trúc RDF
"danh mục … **bao gồm** A, B, C" (một danh sách)	**collection** `( :a :b :c )` → bung thành rdf:first / rdf:rest / rdf:nil
"đạt **điểm 10** học phần này" (quan hệ 3 ngôi: SV – điểm – học phần)	**blank node n-ngôi** `:score [ :value 10 ; :course :sw_course ]`
"Thầy giáo **nhận xét** A là …" (một câu nói *về* một câu khác)	**reification** `[ a rdf:Statement ; rdf:subject … ; rdf:predicate … ; rdf:object … ]`
"thầy giáo" không có tên	blank node `[ a :Teacher ]`

Cách đếm triple: mỗi cạnh trên đồ thị là 1 triple. Collection 3 phần tử tốn 6 cạnh (3 first + 3 rest). Đề giới hạn 20 cạnh, lời giải dưới đây dùng 19 cạnh.

Quy ước vẽ: **IRI vẽ hình elip có tên**, **blank node vẽ elip rỗng**, **literal vẽ hình chữ nhật**, cạnh là mũi tên có nhãn predicate.

▶ Đáp án

```
@prefix : <http://semweb.edu.vn/exam#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
:sw_course :ref (:foswt :learning_sparql :sw_ontologist) ;
    rdfs:label "The Semantic Web" .
:nva a :Student ;
    :score [ :value 10 ; :course :sw_course ] .
[ a :Teacher ; :course :sw_course ] :comment [ a rdf:Statement ;
    rdf:subject :nva ; rdf:predicate rdf:type ; rdf:object :ExcellentStudent ] .
```



Giải thích để ăn điểm:

- `:ref ( … )`: danh sách sách tham khảo là **đóng và có thứ tự**, nên dùng collection. Đồ thị vẽ thành chuỗi blank node nối bằng `rdf:first` / `rdf:rest`, kết thúc ở `rdf:nil`.
- `:score [ :value 10 ; :course :sw_course ]`: điểm là quan hệ 3 ngôi (sinh viên, giá trị, học phần), nên dùng một blank node trung gian.
- Nhận xét của thầy là một mệnh đề nói **về mệnh đề** "nva là ExcellentStudent", nên dùng reification `rdf:Statement`. Thầy là blank node vì đề không cho tên.
- Nếu muốn ghi rõ tên sách, có thể thêm `:foswt rdfs:label "Foundations of Semantic Web Technologies"`. Làm vậy thì tốn thêm cạnh, phải đếm lại để không vượt 20 cạnh.

■ 2022-1 · Câu 1 — Đồ thị RDF + Turtle (sách Design Patterns)

> ***Dạng:*** *D1* · ***Nguồn:*** *2022-C1*

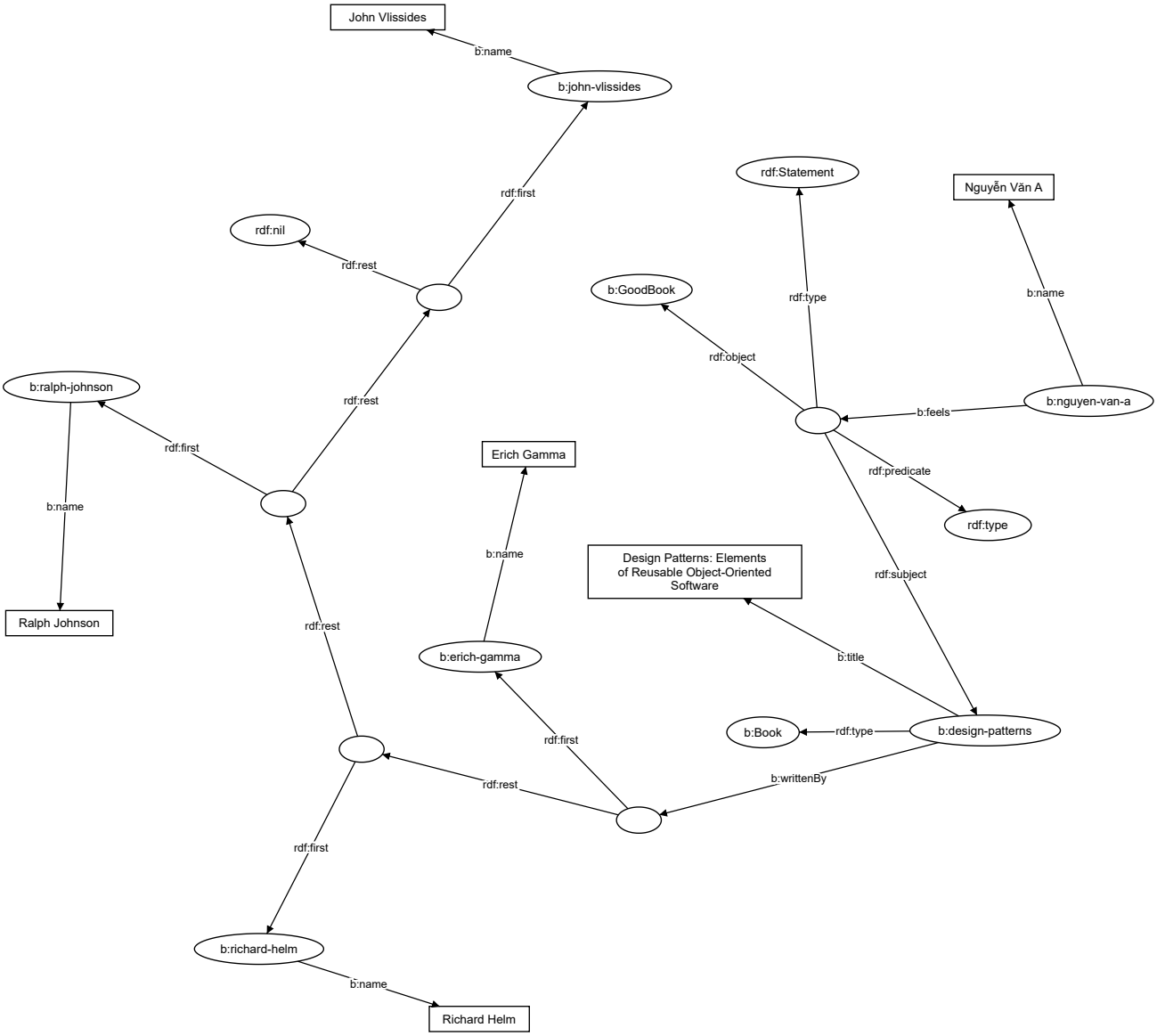
Vẽ đồ thị RDF và viết các bộ-3 theo cú pháp Turtle biểu diễn thông tin trong mô tả sau: Sách Design Patterns: Elements of Reusable Object-Oriented Software được viết bởi 1 nhóm tác giả bao gồm Erich Gamma, Richard Helm, Ralph Johnson và John Vlissides. Nguyễn Văn A cảm thấy đây là 1 cuốn sách hay.

◇ Gợi ý / Khuôn

- "**nhóm tác giả bao gồm** …" là danh sách có thứ tự, đóng, nên dùng collection `( … )`.
- "**A cảm thấy** đây là 1 cuốn sách hay" là ý kiến của A về một mệnh đề, nên dùng reification: mệnh đề (sách, rdf:type, :GoodBook) được bọc trong một `rdf:Statement`.
- Mỗi người được đặt một IRI và gắn `b:name` dạng literal. Sách được gắn `a b:Book` và `b:title`.

▶ Đáp án

```
@prefix b: <http://exam.hust.edu.vn/book/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
b:nguyen-van-a b:name "Nguyễn Văn A" ;
    b:feels [ a rdf:Statement ;
              rdf:subject b:design-patterns ;
              rdf:predicate rdf:type ;
              rdf:object b:GoodBook ] .
b:design-patterns a b:Book ;
    b:title "Design Patterns: Elements of Reusable Object-Oriented Software" ;
    b:writtenBy (b:erich-gamma b:richard-helm b:ralph-johnson b:john-vlissides) .
b:erich-gamma b:name "Erich Gamma" .
b:richard-helm b:name "Richard Helm" .
b:ralph-johnson b:name "Ralph Johnson" .
b:john-vlissides b:name "John Vlissides" .
```



Trên đồ thị, `b:writtenBy` trỏ tới blank node đầu tiên của chuỗi 4 nút. Mỗi nút có `rdf:first` trỏ tới một tác giả và `rdf:rest` trỏ tới nút tiếp theo, nút cuối có `rdf:rest` trỏ tới `rdf:nil`. Mỗi tác giả có thêm cạnh `b:name` trỏ tới một literal (hình chữ nhật).

■ 6390 · Câu 1 — Turtle + đồ thị (C++)

Dạng: *D1* · ***Nguồn:*** *6390-C1*

Hãy tạo biểu diễn RDF bằng Turtle và vẽ đồ thị RDF tương ứng cho những nội dung sau: (a) C++ là 1 ngôn ngữ lập trình, được sáng tạo bởi Bjarne Stroustrup. (b) An (1 sinh viên giả định) nghĩ rằng C++ rất thú vị và tương đối dễ học. (c) Theo chỉ số TIOBE C, C++, Python, Java đang là những ngôn ngữ được sử dụng phổ biến nhất.

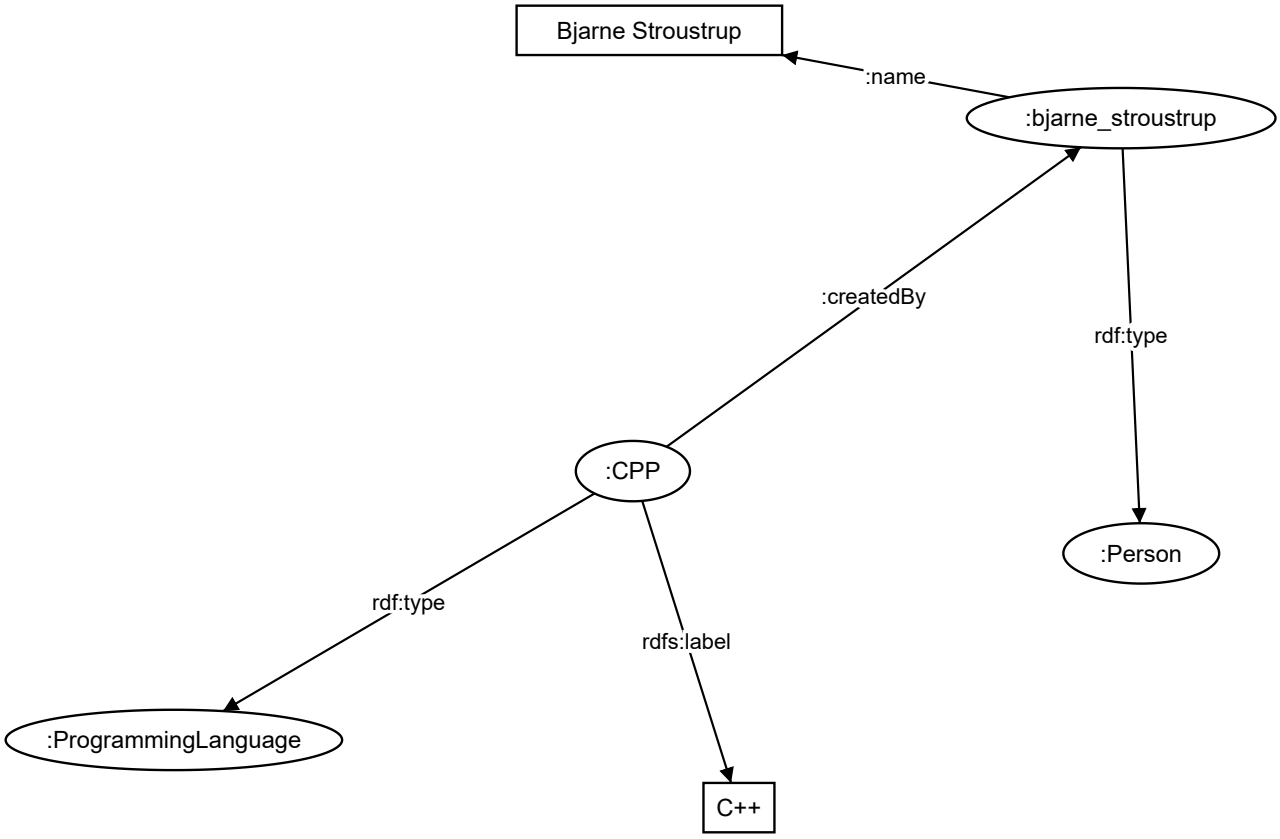
◇ Gợi ý / Khuôn

- (a) Có 2 sự kiện đơn: kiểu của C++ và người tạo ra C++. Nên thêm **kiểu cho mọi thực thể** (Bjarne a :Person) và **nhãn/tên dạng literal**. Theo ghi chú của bạn: liệt kê **đủ** các triple.
- (b) "An **nghĩ rằng** …" là ý kiến về các mệnh đề, nên dùng **reification**. Có 2 ý ("thú vị", "dễ học"), mỗi ý là một `rdf:Statement`.
- (c) Danh sách **có thứ tự** (xếp hạng) lấy từ một **nguồn** (TIOBE), nên dùng collection `( :C :CPP :Python :Java )` gắn vào một nút bảng xếp hạng. Cách khác: dùng `rdf:Seq` với rdf:_1 … rdf:_4.

▶ Đáp án

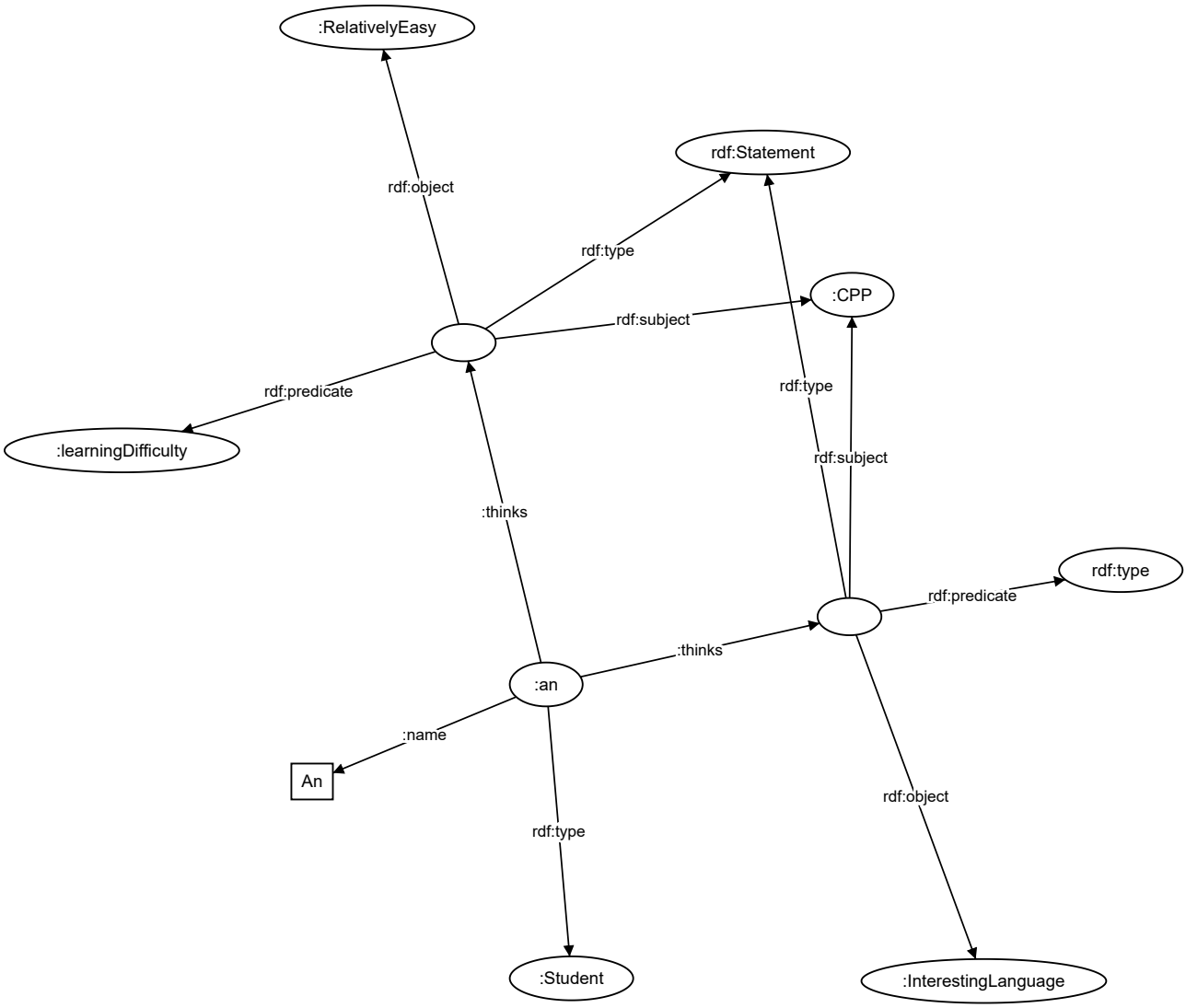
(a)

```
@prefix : <http://semweb.edu.vn/exam#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
:CPP a :ProgrammingLanguage ;
    rdfs:label "C++" ;
    :createdBy :bjarne_stroustrup .
:bjarne_stroustrup a :Person ;
    :name "Bjarne Stroustrup" .
```



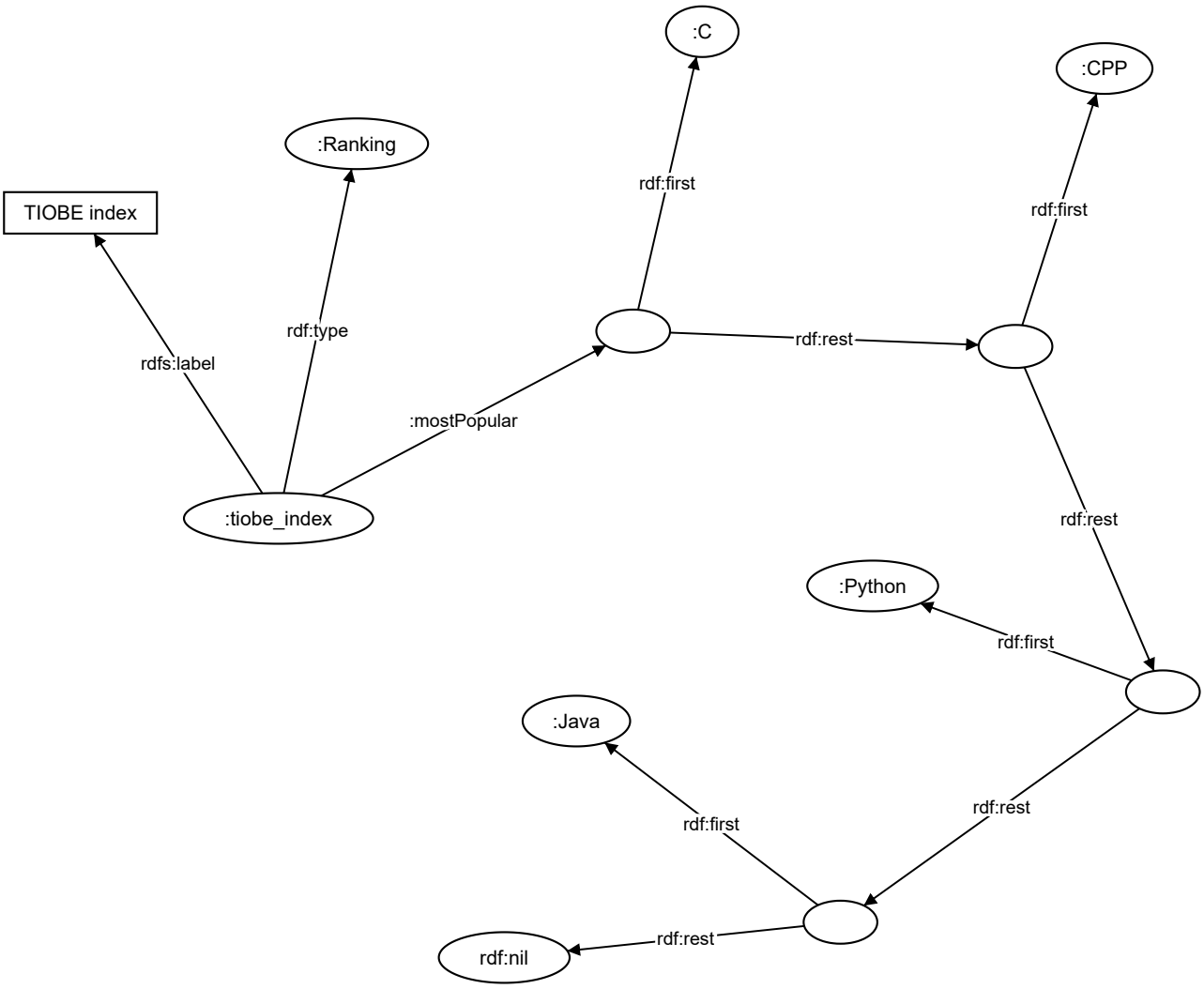
(b)

```
@prefix : <http://semweb.edu.vn/exam#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
:an a :Student ; :name "An" ;
    :thinks [ a rdf:Statement ;
              rdf:subject :CPP ; rdf:predicate rdf:type ; rdf:object :InterestingLanguage ] ,
            [ a rdf:Statement ;
              rdf:subject :CPP ; rdf:predicate :learningDifficulty ; rdf:object :RelativelyEasy
] .
```



(c)

```
@prefix : <http://semweb.edu.vn/exam#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
:tiobe_index a :Ranking ;
    rdfs:label "TIOBE index" ;
    :mostPopular (:C :CPP :Python :Java) .
```



Giải thích:

- (b) không được viết thẳng `:CPP a :InterestingLanguage`. Viết như vậy là khẳng định một **sự thật**, trong khi đề chỉ nói đó là **ý kiến của An**. Phải dùng reification.
- (c) collection giữ được **thứ tự** xếp hạng và là danh sách **đóng** (đúng 4 ngôn ngữ). Nếu muốn có thể thêm `:C a :ProgrammingLanguage .` cho cả 4 ngôn ngữ.

Cách viết khác cho (c) bằng container:

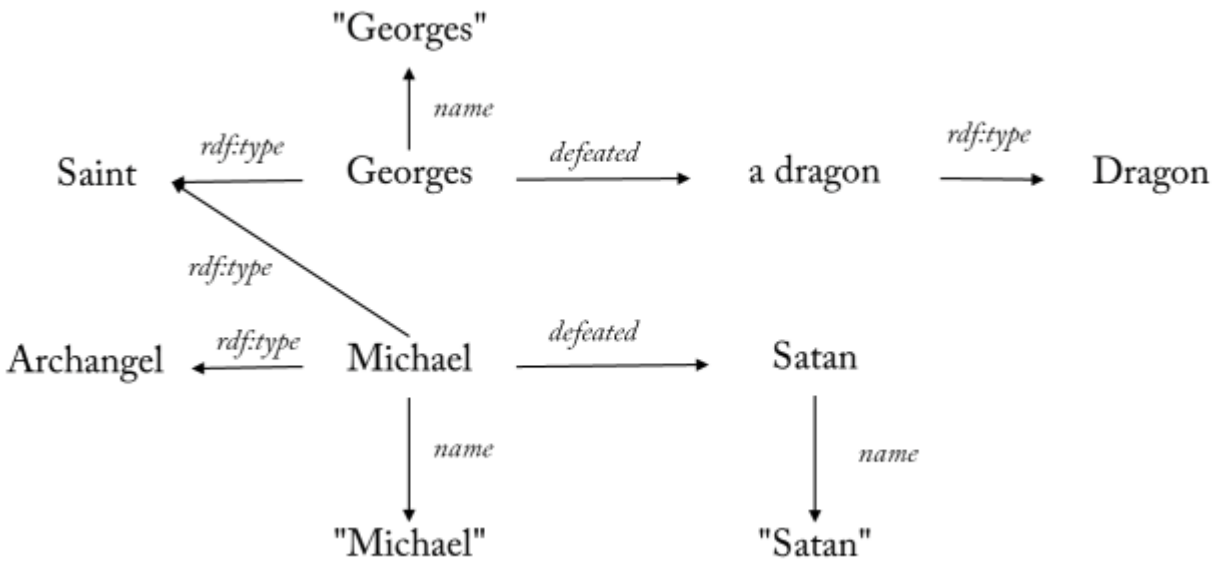
```
:tiobe_index :mostPopular [ a rdf:Seq ; rdf:_1 :C ; rdf:_2 :CPP ; rdf:_3 :Python ; rdf:_4 :Java
] .
```

D2 Chuyển đổi đồ thị ↔ Turtle ↔ N-Triples ↔ RDF/XML

Đề cho đồ thị → viết Turtle / N-Triples / RDF-XML, hoặc ngược lại.

③ Bài tập tuần — lời giải mẫu

■ Part 1 Ex4 — Đồ thị Georges/Michael → Turtle




```
@prefix :     <http://example.org/> .
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .

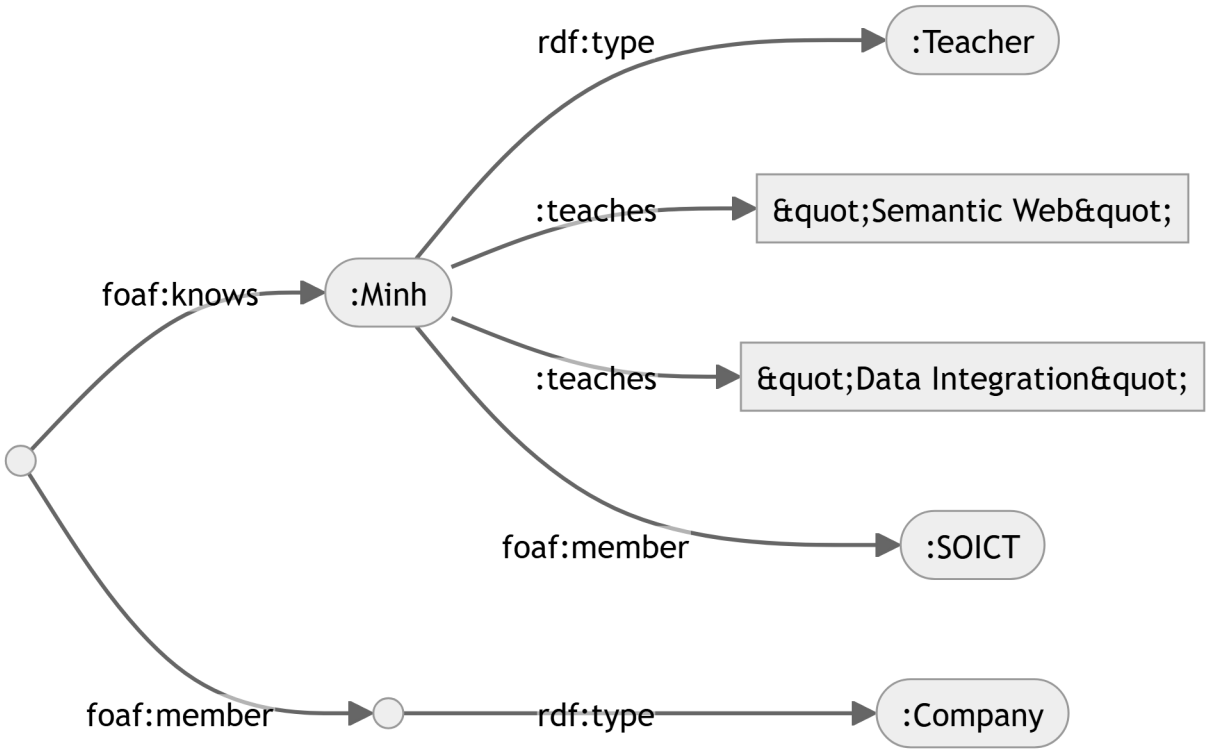
:Georges rdf:type :Saint ;
    foaf:name "Georges" ;
    :defeated [ rdf:type :Dragon ] .        # "a dragon" = blank node (không có tên)
:Michael rdf:type :Saint , :Archangel ;
    foaf:name "Michael" ;
    :defeated :Satan .
:Satan foaf:name "Satan" .
```

Bẫy: nút "a dragon" viết thường, không phải URI → **blank node**; Michael có **hai** `rdf:type`. Đồ thị có **9 triple**.

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
         xmlns:ex="http://example.org/">
  <ex:Saint rdf:about="http://example.org/Georges">
    <foaf:name>Georges</foaf:name>
    <ex:defeated>
      <ex:Dragon/>
    </ex:defeated>
  </ex:Saint>
  <ex:Saint rdf:about="http://example.org/Michael">
    <rdf:type rdf:resource="http://example.org/Archangel"/>
    <foaf:name>Michael</foaf:name>
    <ex:defeated>
      <rdf:Description rdf:about="http://example.org/Satan">
        <foaf:name>Satan</foaf:name>
      </rdf:Description>
    </ex:defeated>
  </ex:Saint>
</rdf:RDF>
```

(`<ex:Dragon/>` lồng không có `rdf:about` = blank node có kiểu Dragon — typed node element.)

■ Part 1 Ex6 — Turtle → đồ thị



Đọc thành lời: *Minh là giáo viên, dạy "Semantic Web" và "Data Integration", là thành viên SOICT. Có một người (không rõ là ai) quen Minh và là thành viên của một công ty nào đó.* — **7 triple**, 2 blank node.

N-Triples:

```
<http://example.org/Minh> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>
<http://example.org/Teacher> .
<http://example.org/Minh> <http://example.org/teaches> "Semantic Web" .
<http://example.org/Minh> <http://example.org/teaches> "Data Integration" .
<http://example.org/Minh> <http://xmlns.com/foaf/0.1/member> <http://example.org/SOICT> .
_:b1 <http://xmlns.com/foaf/0.1/knows> <http://example.org/Minh> .
_:b1 <http://xmlns.com/foaf/0.1/member> _:b2 .
_:b2 <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://example.org/Company> .
```

D3 RDFS: schema (lớp, thuộc tính, domain, range)

Đề bắt khai báo lớp, thuộc tính, domain/range bằng RDFS và vẽ đồ thị schema.

③ Bài tập tuần — lời giải mẫu

■ Part 1 Ex3 — URI1…URI6

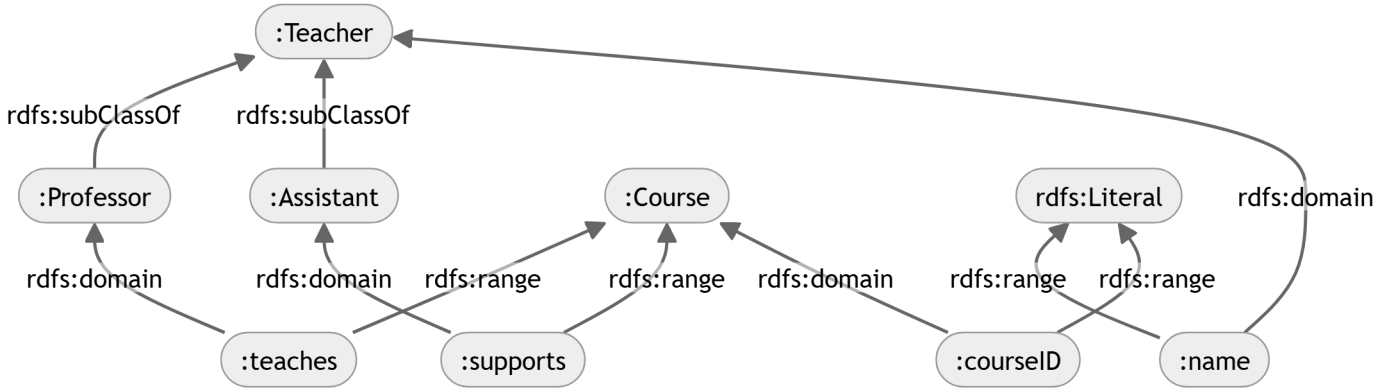
```
@prefix ex:   <http://example.org/> .
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

ex:URI1 a rdfs:Class .
ex:URI2 a rdfs:Class .
ex:URI3 a rdf:Property ;
    rdfs:domain ex:URI1 ;
    rdfs:range  ex:URI2 .
ex:URI4 a ex:URI1 .
ex:URI5 a ex:URI2 .
ex:URI6 a ex:URI2 .
```

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#">
  <rdfs:Class rdf:about="http://example.org/URI1"/>
  <rdfs:Class rdf:about="http://example.org/URI2"/>
  <rdf:Property rdf:about="http://example.org/URI3">
    <rdfs:domain rdf:resource="http://example.org/URI1"/>
    <rdfs:range rdf:resource="http://example.org/URI2"/>
  </rdf:Property>
  <rdf:Description rdf:about="http://example.org/URI4">
    <rdf:type rdf:resource="http://example.org/URI1"/>
  </rdf:Description>
  <rdf:Description rdf:about="http://example.org/URI5">
    <rdf:type rdf:resource="http://example.org/URI2"/>
  </rdf:Description>
  <rdf:Description rdf:about="http://example.org/URI6">
    <rdf:type rdf:resource="http://example.org/URI2"/>
  </rdf:Description>
</rdf:RDF>
```

Câu hỏi mở rộng hay gặp: thêm `ex:URI5 ex:URI3 ex:URI6 .` thì suy ra gì? → `URI5 a URI1` (rdfs2), `URI6 a URI2` (rdfs3, đã có). Không mâu thuẫn dù URI5 vừa là URI1 vừa là URI2 (RDFS không có disjoint).

■ Part 1 Ex5 — Teacher/Professor/Assistant/Course (D3)




```
@prefix :     <http://example.org/school#> .
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

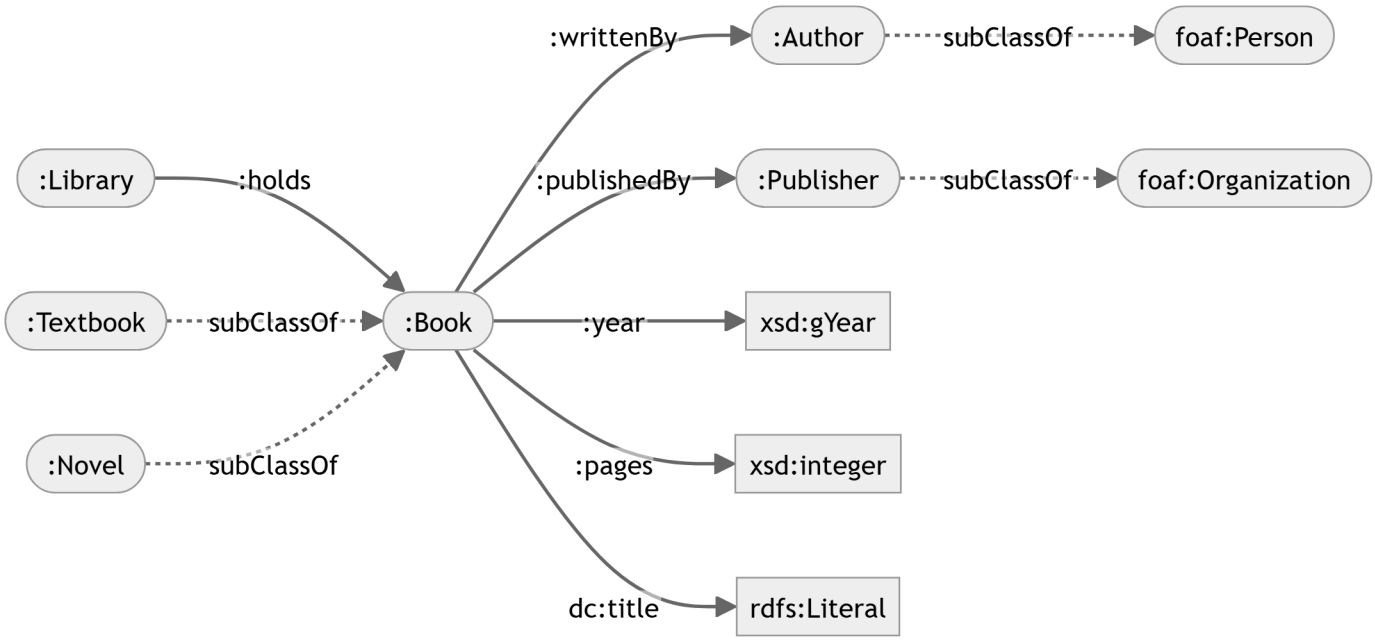
:Teacher   a rdfs:Class .
:Professor a rdfs:Class ; rdfs:subClassOf :Teacher .
:Assistant a rdfs:Class ; rdfs:subClassOf :Teacher .
:Course    a rdfs:Class .

:name     a rdf:Property ; rdfs:domain :Teacher ; rdfs:range rdfs:Literal .
:courseID a rdf:Property ; rdfs:domain :Course  ; rdfs:range rdfs:Literal .
:teaches  a rdf:Property ; rdfs:domain :Professor ; rdfs:range :Course .
:supports a rdf:Property ; rdfs:domain :Assistant ; rdfs:range :Course .   # "supports the course
management"
```

Nhận xét ăn điểm: "Each Teacher has a name" / "Each Course has a courseID" là ràng buộc **bắt buộc có** — RDFS **không** diễn đạt được (domain chỉ nói: *ai có* name thì là Teacher). Cần OWL: `Teacher ⊑ ∃name.xsd:string` (hoặc `min 1`). Tương tự, domain `teaches` = Professor nghĩa là *ai dạy thì là Professor* ⇒ nếu Assistant cũng `teaches` thì bị suy ra là Professor (side-effect của domain toàn cục).

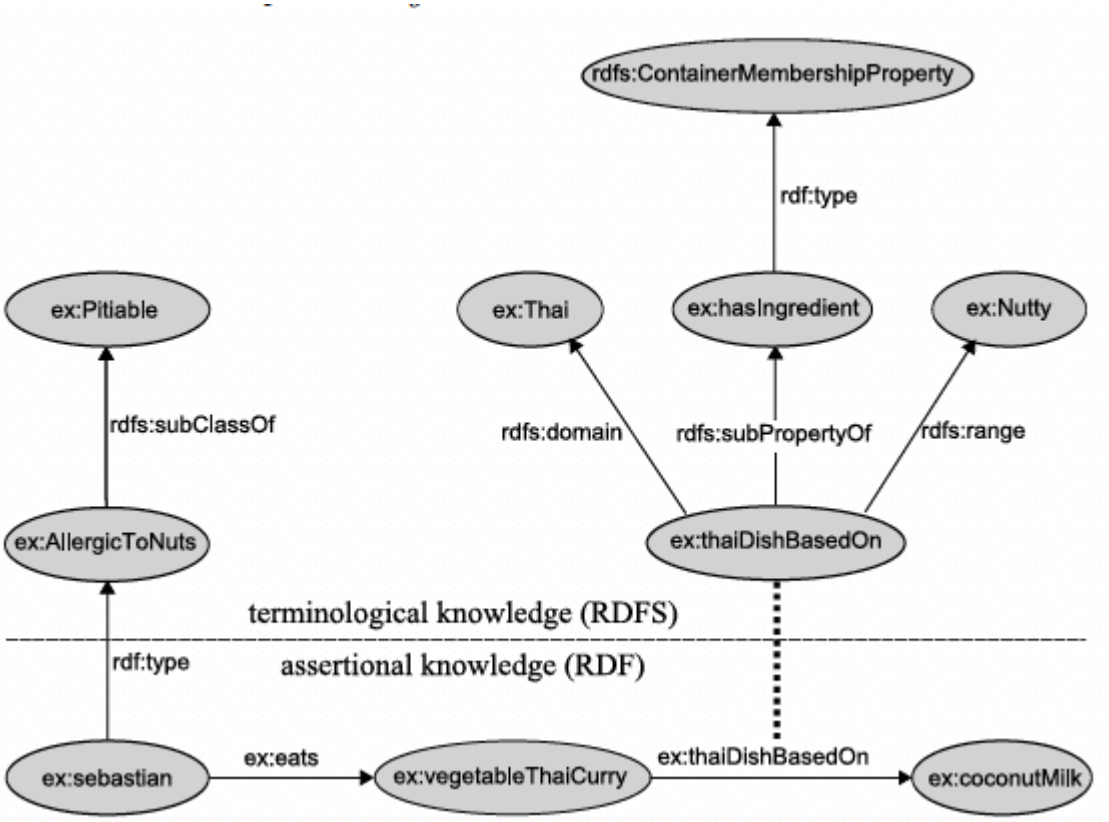
■ Part 1 Ex7 — Thư viện (D3, đề mở)

Dữ liệu đầy đủ, chạy được: dữ liệu `library`. Sơ đồ schema:



Gợi ý chấm: có phân cấp lớp; property có domain/range; dùng lại vocab chuẩn (`dc:title`, `foaf:name`); literal có datatype (`"2012"^^xsd:gYear`) và lang tag (`"Truyện Kiều"@vi`); ít nhất một ví dụ nhiều tác giả.

■ Part 2 Ex1 — Mô tả ontology trong hình




```
@prefix ex:   <http://example.org/> .
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

# T-Box (terminological knowledge)
ex:AllergicToNuts rdfs:subClassOf ex:Pitiable .
ex:hasIngredient  rdf:type rdfs:ContainerMembershipProperty .
ex:thaiDishBasedOn rdfs:subPropertyOf ex:hasIngredient ;
    rdfs:domain ex:Thai ;
    rdfs:range  ex:Nutty .
# A-Box (assertional knowledge)
ex:sebastian rdf:type ex:AllergicToNuts ;
    ex:eats ex:vegetableThaiCurry .
ex:vegetableThaiCurry ex:thaiDishBasedOn ex:coconutMilk .
```

- **5 class:** Pitiable, AllergicToNuts, Thai, Nutty, rdfs:ContainerMembershipProperty · **properties:** hasIngredient, thaiDishBasedOn (+ eats ở A-Box) · **instances:** sebastian, vegetableThaiCurry, coconutMilk.
- **Suy diễn:** `sebastian a Pitiable` (rdfs9) · `vegetableThaiCurry a Thai` (rdfs2) · `coconutMilk a Nutty` (rdfs3) · `vegetableThaiCurry hasIngredient coconutMilk` (rdfs7) · `hasIngredient subPropertyOf rdfs:member` (rdfs12) ⇒ `vegetableThaiCurry rdfs:member coconutMilk` (rdfs7).
- Ý nghĩa: Sebastian dị ứng hạt, ăn cà ri Thái làm từ nước cốt dừa (một thứ "Nutty") ⇒ đáng thương (Pitiable). Đây là ví dụ kinh điển trong sách Hitzler et al., *Foundations of Semantic Web Technologies*.

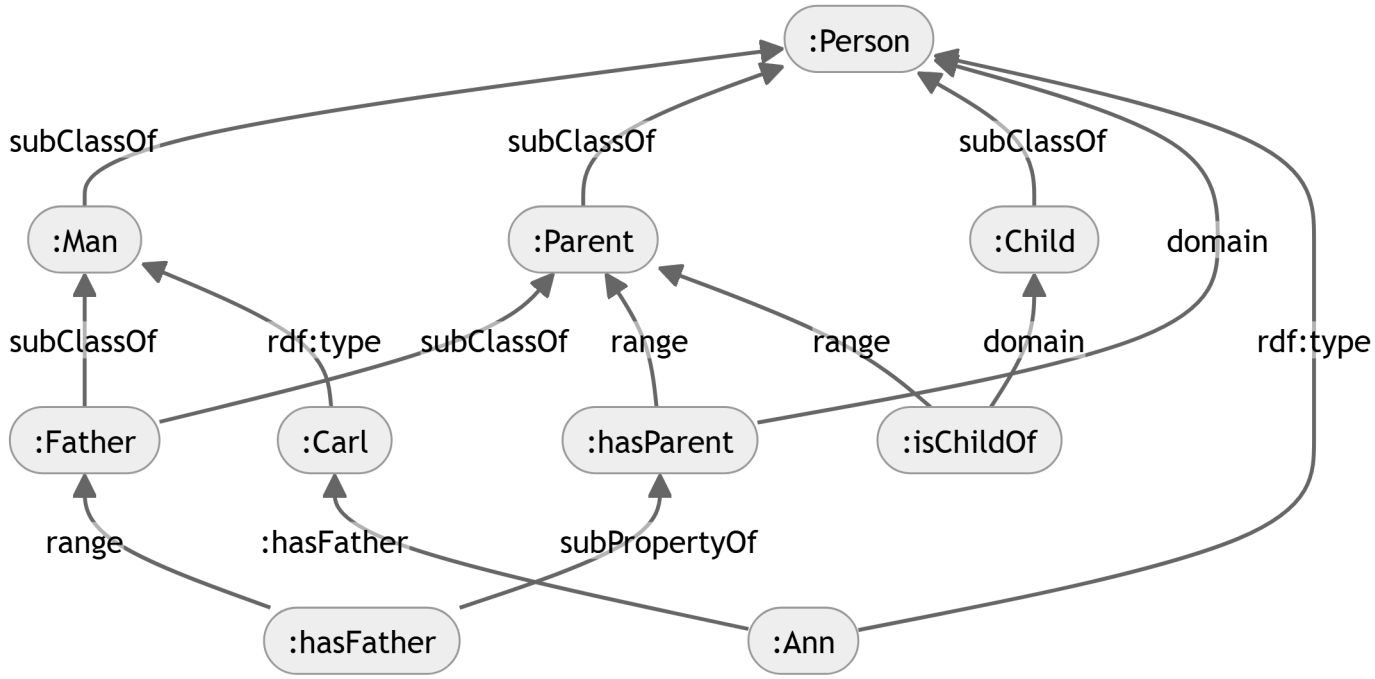
D4 Suy diễn RDFS: luật + cơ sở, true / unknown

Đề cho một đồ thị → viết các triple suy ra được (kèm tên luật) hoặc trả lời đúng / không xác định.

③ Bài tập tuần (HW 5.9, 12.9)

■ Part 2 Ex2 — Đồ thị gia đình (Person/Father/Ann/Carl)

A. Đồ thị



(Ngoài ra mỗi class có `rdf:type rdfs:Class`, mỗi property có `rdf:type rdf:Property` — khi thi vẽ đủ hoặc ghi chú "các nút class đều có rdf:type rdfs:Class".)

B. Bao đóng (chỉ phần dùng tới)

- (T1) `:Ann :hasParent :Carl` — rdfs7 (hasFather ⊑ hasParent)
- (T2) `:Carl a :Father` — rdfs3 (range hasFather)
- (T3) `:Carl a :Parent` — rdfs9 (T2 + Father ⊑ Parent); cũng từ rdfs3 với range hasParent
- (T4) `:Carl a :Person` — rdfs9 (Man ⊑ Person)
- (T5) `:Ann a :Person` — rdfs2 (domain hasParent) + có sẵn
- (T6) `:Father rdfs:subClassOf :Person` — rdfs11 (Father ⊑ Parent ⊑ Person)
- (T7) `:Person rdfs:subClassOf :Person` — rdfs10
- (T8) `:Child a rdfs:Resource`, `rdfs:range a rdfs:Resource` — rdfs4a

Bảng 18 mệnh đề (đã kiểm bằng máy)

#	Mệnh đề	Kết quả	Lý do
1	`:Father rdfs:subClassOf :Person`	**TRUE**	rdfs11: Father ⊑ Parent ⊑ Person (T6)
2	`:Man rdfs:subClassOf :Person`	**TRUE**	có sẵn trong G
3	`:Carl a :Person`	**TRUE**	rdfs9 (T4)
4	`:Carl a :Parent`	**TRUE**	rdfs3 + rdfs9 (T3)

#	Mệnh đề	Kết quả	Lý do
5	`:Carl :hasChild :Ann`	UNKNOWN	`:hasChild` không xuất hiện trong G — RDFS không có inverse; các luật RDFS không tạo ra IRI mới, nên không luật nào sinh được triple có predicate `:hasChild`
6	`:Carl a :Man`	**TRUE**	có sẵn
7	`:Carl a :Father`	**TRUE**	rdfs3 (T2)
8	`:Child rdf:type rdfs:Resource`	**TRUE**	rdfs4a (Child là subject của triple)
9	`:Ann a :Child`	UNKNOWN	chỉ suy ra được nếu Ann là subject của `:isChildOf` (rdfs2) — không có
10	`:Ann :isChildOf :Carl`	UNKNOWN	`isChildOf` chỉ xuất hiện trong schema, không có subPropertyOf nối với hasParent/hasFather; tên "nghịch đảo" không mang nghĩa hình thức
11	`:Ann :hasParent :Carl`	**TRUE**	rdfs7 (T1)
12	`:Ann :hasParent _:x`	**TRUE**	blank node = biến tồn tại: `_:x ↦ :Carl` cho T1 (luật se1: thay object bằng blank node)
13	`:Ann :hasParent [ rdf:type :Person ]`	**TRUE**	`_:b ↦ :Carl`: T1 và T4 cùng đúng
14	`:hasFather rdfs:domain :Person`	UNKNOWN	không luật RDFS nào sinh ra bộ-3 `subP rdfs:domain C` (rdfs2 chỉ *dùng* domain có sẵn). Ở mức cá thể thì domain vẫn "truyền": `:Ann :hasFather :Carl` ⇒ rdfs7 ⇒ rdfs2 ⇒ `:Ann a :Person` (slide: *"inherited by sub properties"*) — nhưng mệnh đề hỏi bộ-3 schema
15	`rdfs:range rdf:type rdfs:Resource`	**TRUE**	rdfs4a trên axiomatic triple `rdfs:range rdfs:domain rdf:Property`
16	`:hasFather rdfs:range :Father`	**TRUE**	**có sẵn trong G** (`:hasFather … rdfs:range :Father`)
17	`:hasFather rdfs:domain [ rdfs:subClassOf :Person ]`	UNKNOWN	không suy ra được bộ-3 `rdfs:domain` nào cho hasFather (cùng lý do mệnh đề 14 — bộ-3 schema không được sinh, dù ở mức cá thể domain vẫn "truyền") ⇒ không có ứng viên cho blank node
18	`:Father rdfs:subClassOf [ rdfs:subClassOf :Person ]`	**TRUE**	`_:b ↦ :Person`: T6 + T7 (rdfs10) — hoặc `_:b ↦ :Parent` (Father ⊑ Parent có sẵn, Parent ⊑ Person có sẵn)

> *(!)* ***Đính chính bài về nhà 12.9 (câu 3 = mệnh đề 16):*** *bài nộp ghi Unknown với lý do "No range is declared for :hasFather in G" —* ***sai****: đề gốc có* `:hasFather a rdf:Property ; rdfs:subPropertyOf :hasParent ; rdfs:range :Father .` *⇒ mệnh đề có sẵn trong G ⇒* ***TRUE*** *(bài 5.9 cũng dùng triple này ở bước T2). Lý luận "bộ-3* `rdfs:range` *không suy ra được từ superproperty" vẫn đúng nhưng không áp dụng ở đây. (!)* ***Đính chính nhỏ bài 5.9:*** *luật "thay IRI ở vị trí* ***object*** *bằng blank node" trong RDF 1.0 Semantics tên là* ***se1****;* ***se2*** *là thay ở vị trí* ***subject****. (RDF 1.1 không đặt tên luật, dùng Interpolation Lemma.)*

---

④ Đề thi cũ dạng này

■ 2022-1 · Câu 2 — Suy diễn RDFS

> ***Dạng:*** *D4 ·* ***Nguồn:*** *2022-C2*

Hãy viết 4 bộ-3 có thể được suy ra dựa trên CSTT như sau trong phạm vi ngữ nghĩa RDFS và giải thích các bộ-3 là cơ sở của các suy diễn đó:

```
@prefix : <http://exam.hust.edu.vn/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
:a :r :b.
:c a :C.
:r rdfs:range :C; rdfs:domain :D.
:C rdfs:subClassOf :E.
:D rdfs:subClassOf :F.
```

◇ Gợi ý / Khuôn

Viết theo mẫu `triple suy ra <= triple cơ sở 1. triple cơ sở 2. (tên luật)`. Các luật hay dùng:

- **rdfs2 (domain):** `x p y` và `p domain D` cho ra `x a D`.
- **rdfs3 (range):** `x p y` và `p range R` cho ra `y a R`.
- **rdfs9 (subClass):** `x a C` và `C subClassOf E` cho ra `x a E`.

Đề yêu cầu 4 triple, nhưng nên viết cả 5 để chắc điểm.

▶ Đáp án

```
:a a :D.  <=  :a :r :b.   :r rdfs:domain :D.        (rdfs2)
:a a :F.  <=  :a a :D.    :D rdfs:subClassOf :F.    (rdfs9)
:c a :E.  <=  :c a :C.    :C rdfs:subClassOf :E.    (rdfs9)
:b a :C.  <=  :a :r :b.   :r rdfs:range :C.         (rdfs3)
:b a :E.  <=  :b a :C.    :C rdfs:subClassOf :E.    (rdfs9)
```

Các triple suy ra được nhưng tầm thường (vẫn đúng, chỉ nên ghi thêm): `:r a rdf:Property` (rdf1), `:C a rdfs:Class` (rdfs2 + tiên đề `rdfs:subClassOf rdfs:domain rdfs:Class`), `:C rdfs:subClassOf :C` (rdfs10).

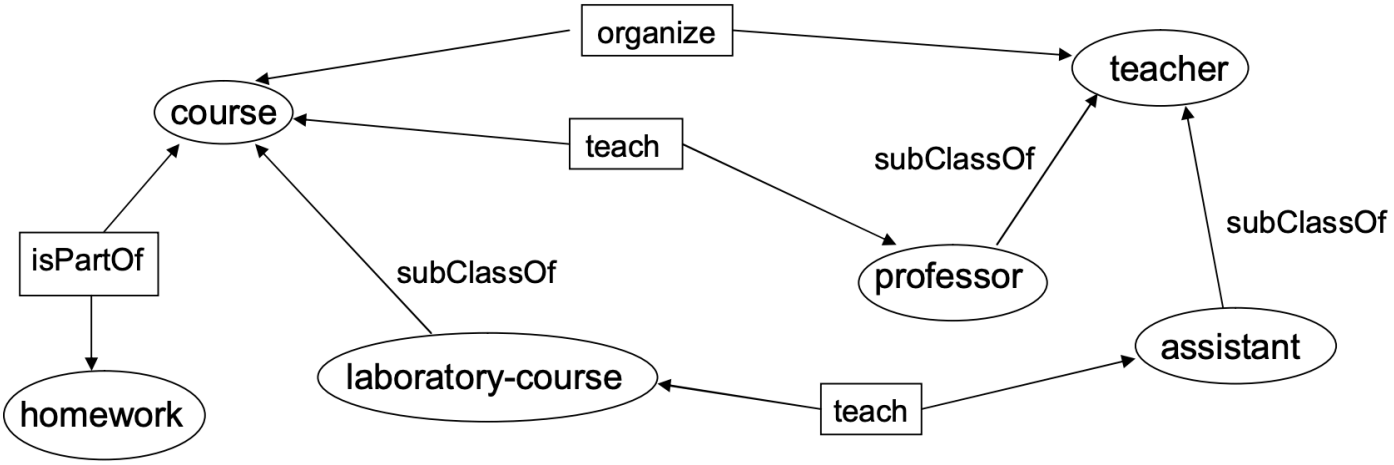
D5 Thiết kế ontology OWL + biểu thức DL

Đề cho câu mô tả → viết tiên đề OWL (Turtle), biểu thức DL (và Manchester nếu đề hỏi).

③ Bài tập tuần (bài 3 laboratory-course, thuốc, phim)

■ Lời giải Part 3 Ex3

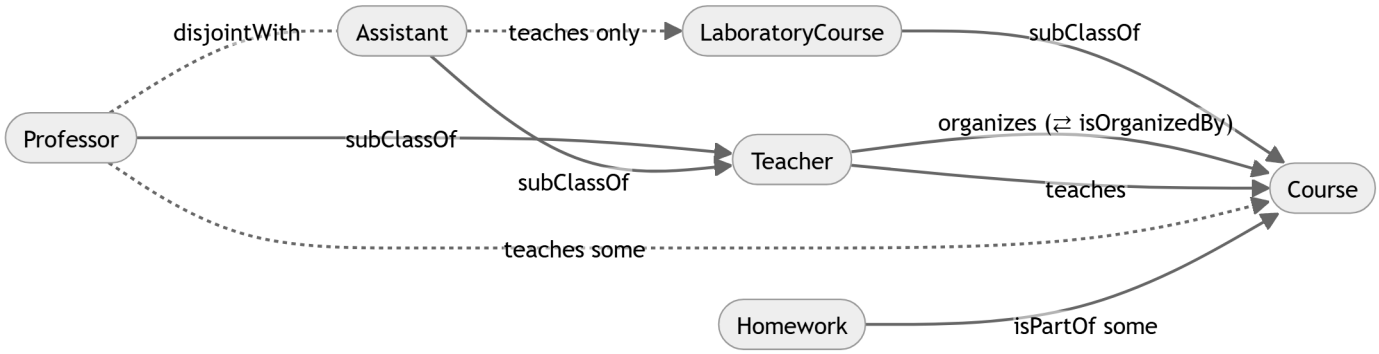
> ***Đề:*** *"A laboratory-course is a course. Courses are organized by teachers. Teachers can be professors or assistants. The professor teaches the courses, while the assistant only teaches the practical courses. Homework is part of the course."*



Câu đề → tiên đề

#	Câu	Tiên đề	Ghi chú
1	A laboratory-course is a course	`LaboratoryCourse ⊑ Course`	"practical course" = laboratory-course
2	Courses are organized by teachers	`organizes`: domain Teacher, range Course; `isOrganizedBy ≡ organizes⁻`; `Course ⊑ ∃isOrganizedBy.Teacher`	"are organized" ⇒ mỗi course có ≥1 người tổ chức
3	Teachers can be professors or assistants	`Professor ⊑ Teacher`, `Assistant ⊑ Teacher`; *(giả định)* `Teacher ≡ Professor ⊔ Assistant`, `Professor ⊓ Assistant ⊑ ⊥`	"can be" — tối thiểu là 2 subClassOf; covering + disjoint là giả định hợp lý, **ghi rõ trong bài**
4	The professor teaches the courses	`teaches`: domain Teacher, range Course; `Professor ⊑ ∃teaches.Course`	
5	the assistant **only** teaches the practical courses	`Assistant ⊑ ∀teaches.LaboratoryCourse`	**từ khóa "only" ⇒ allValuesFrom** — đây là điểm mấu chốt của bài
6	Homework is part of the course	`Homework ⊑ ∃isPartOf.Course`; `isPartOf`: domain Homework, range Course	có thể thêm `hasPart ≡ isPartOf⁻`
+	(thiết kế tốt)	`AllDisjointClasses(Course, Teacher, Homework)`	để reasoner phát hiện dùng sai

Sơ đồ



DL

```
T-Box:
  LaboratoryCourse ⊑ Course
  Professor ⊑ Teacher            Assistant ⊑ Teacher
  Teacher ≡ Professor ⊔ Assistant          (giả định: covering)
  Professor ⊓ Assistant ⊑ ⊥                (giả định: disjoint)
  Course ⊑ ∃isOrganizedBy.Teacher          isOrganizedBy ≡ organizes⁻
  ∃organizes.⊤ ⊑ Teacher     ⊤ ⊑ ∀organizes.Course
  ∃teaches.⊤ ⊑ Teacher       ⊤ ⊑ ∀teaches.Course
  Professor ⊑ ∃teaches.Course
  Assistant ⊑ ∀teaches.LaboratoryCourse
  Homework ⊑ ∃isPartOf.Course
  ∃isPartOf.⊤ ⊑ Homework     ⊤ ⊑ ∀isPartOf.Course
  Course ⊓ Teacher ⊑ ⊥   Course ⊓ Homework ⊑ ⊥   Teacher ⊓ Homework ⊑ ⊥
A-Box (ví dụ):
  Professor(hung), teaches(hung, SemanticWeb), Course(SemanticWeb)
  Assistant(minh), teaches(minh, SemanticWebLab), LaboratoryCourse(SemanticWebLab)
  Homework(hw1), isPartOf(hw1, SemanticWeb)
```

Manchester

```
Class: Course
    SubClassOf: isOrganizedBy some Teacher
Class: LaboratoryCourse
    SubClassOf: Course
Class: Teacher
    EquivalentTo: Professor or Assistant
Class: Professor
    SubClassOf: Teacher, teaches some Course
    DisjointWith: Assistant
Class: Assistant
    SubClassOf: Teacher, teaches only LaboratoryCourse
Class: Homework
    SubClassOf: isPartOf some Course
DisjointClasses: Course, Teacher, Homework

ObjectProperty: organizes     Domain: Teacher    Range: Course
ObjectProperty: isOrganizedBy InverseOf: organizes
ObjectProperty: teaches       Domain: Teacher    Range: Course
ObjectProperty: isPartOf      Domain: Homework   Range: Course
ObjectProperty: hasPart       InverseOf: isPartOf
```

Turtle (phần T-Box — bản đầy đủ kèm dữ liệu ở dữ liệu `ex3-lab-course`)

```
@prefix :     <http://example.org/course#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

:Course a owl:Class ;
    rdfs:subClassOf [ a owl:Restriction ; owl:onProperty :isOrganizedBy ; owl:someValuesFrom
:Teacher ] .
:LaboratoryCourse a owl:Class ; rdfs:subClassOf :Course .
:Teacher a owl:Class ;
    owl:equivalentClass [ a owl:Class ; owl:unionOf ( :Professor :Assistant ) ] .
:Professor a owl:Class ;
    rdfs:subClassOf :Teacher ,
        [ a owl:Restriction ; owl:onProperty :teaches ; owl:someValuesFrom :Course ] ;
    owl:disjointWith :Assistant .
:Assistant a owl:Class ;
    rdfs:subClassOf :Teacher ,
        [ a owl:Restriction ; owl:onProperty :teaches ; owl:allValuesFrom :LaboratoryCourse ] .
:Homework a owl:Class ;
    rdfs:subClassOf [ a owl:Restriction ; owl:onProperty :isPartOf ; owl:someValuesFrom :Course ]
.
[] a owl:AllDisjointClasses ; owl:members ( :Course :Teacher :Homework ) .

:organizes     a owl:ObjectProperty ; rdfs:domain :Teacher ; rdfs:range :Course .
:isOrganizedBy a owl:ObjectProperty ; owl:inverseOf :organizes .
:teaches       a owl:ObjectProperty ; rdfs:domain :Teacher ; rdfs:range :Course .
:isPartOf      a owl:ObjectProperty ; rdfs:domain :Homework ; rdfs:range :Course .
:hasPart       a owl:ObjectProperty ; owl:inverseOf :isPartOf .
```

Giải thích lựa chọn (phần "ăn điểm")

- **Vì sao `only` chứ không phải range?** Nếu viết `teaches rdfs:range LaboratoryCourse` thì **giáo sư** cũng chỉ dạy được môn thực hành — sai. Restriction `Assistant ⊑ ∀teaches.LaboratoryCourse` là **cục bộ** cho Assistant.
- **Vì sao Professor dùng `some`?** "The professor teaches the courses" = giáo sư (có) dạy môn học — khẳng định tồn tại; không giới hạn chỉ dạy loại nào.
- **Có cần `Assistant ⊑ ∃teaches.LaboratoryCourse` không?** Đề không nói trợ giảng *bắt buộc* phải dạy; `only` một mình cho phép trợ giảng không dạy gì. Nếu muốn "trợ giảng dạy (ít nhất) một môn thực hành và chỉ môn thực hành" thì thêm `some` — nêu như một lựa chọn.
- **Covering axiom** `Teacher ≡ Professor ⊔ Assistant`: "can be" có thể hiểu là "chỉ có thể là" ⇒ thêm; nếu không chắc, ghi là giả định.

Các biến thể có thể ra thi

V1. Trợ giảng Minh dạy môn Semantic Web (một `:Course` thường). Reasoner kết luận gì? Có mâu thuẫn không?

▶ Đáp án

`Assistant(minh)`, `teaches(minh, SemanticWeb)` + `Assistant ⊑ ∀teaches.LaboratoryCourse` ⇒ reasoner **suy ra `LaboratoryCourse(SemanticWeb)`** — **không** báo lỗi (OWA: không có gì nói SemanticWeb *không phải* lab course). Chỉ mâu thuẫn nếu thêm `SemanticWeb a [ owl:complementOf :LaboratoryCourse ]` hoặc một lớp `TheoryCourse` disjoint với LaboratoryCourse. ⇒ **OWL suy diễn, không kiểm tra ràng buộc**; muốn kiểm tra kiểu "đóng" dùng SPARQL (bài q08 (trang D8)) hoặc SHACL.

V2. Thêm `:x :teaches :SemanticWeb .` (không nói x là gì). Suy ra gì về x?

▶ Đáp án

Domain ⇒ `Teacher(x)`; `Teacher ≡ Professor ⊔ Assistant` ⇒ x là Professor **hoặc** Assistant (không biết cái nào). Nếu thêm `¬LaboratoryCourse(SemanticWeb)` thì x không thể là Assistant (∀) ⇒ **x là Professor** — ví dụ đẹp về suy diễn qua phép hợp (tableau tách nhánh, một nhánh đóng).

V3. Bổ sung: "mỗi môn học có ít nhất 2 bài tập về nhà", "mỗi môn thực hành do đúng một trợ giảng phụ trách", "giáo sư không tổ chức quá 3 môn".

▶ Đáp án

`Course ⊑ ≥2 hasPart.Homework`	`Manchester: Course SubClassOf: hasPart min 2 Homework`
`LaboratoryCourse ⊑ =1 isTaughtBy.Assistant`	`LaboratoryCourse SubClassOf: isTaughtBy exactly 1 Assistant`
`Professor ⊑ ≤3 organizes.Course`	`Professor SubClassOf: organizes max 3 Course`

(`isTaughtBy ≡ teaches⁻`.) Turtle dùng `owl:minQualifiedCardinality "2"^^xsd:nonNegativeInteger ; owl:onClass :Homework` (=1 → `owl:qualifiedCardinality`, ≤3 → `owl:maxQualifiedCardinality`, cùng kèm `owl:onClass`). Lưu ý OWA + không có UNA: 2 homework phải `differentFrom` nhau mới được tính là 2; reasoner không báo lỗi khi môn chỉ có 1 homework (có thể còn homework chưa biết).

V4. Định nghĩa lớp: "LabTeacher là giáo viên dạy ít nhất một môn thực hành"; "BusyProfessor là giáo sư dạy ít nhất 3 môn". Ai trong dữ liệu mẫu được xếp vào?

▶ Đáp án

`LabTeacher ≡ Teacher ⊓ ∃teaches.LaboratoryCourse` (Manchester: `Teacher and teaches some LaboratoryCourse`) — phải là **EquivalentTo** để reasoner tự xếp ⇒ minh, an. `BusyProfessor ≡ Professor ⊓ ≥3 teaches.Course` ⇒ không ai (hung dạy 2 môn; và kể cả có 3 tên khác nhau, cần `differentFrom` vì **không có** UNA).

V5 (nối sang tableau). Chứng minh `Assistant ⊓ ∃teaches.Course ⊑ ∃teaches.LaboratoryCourse`.

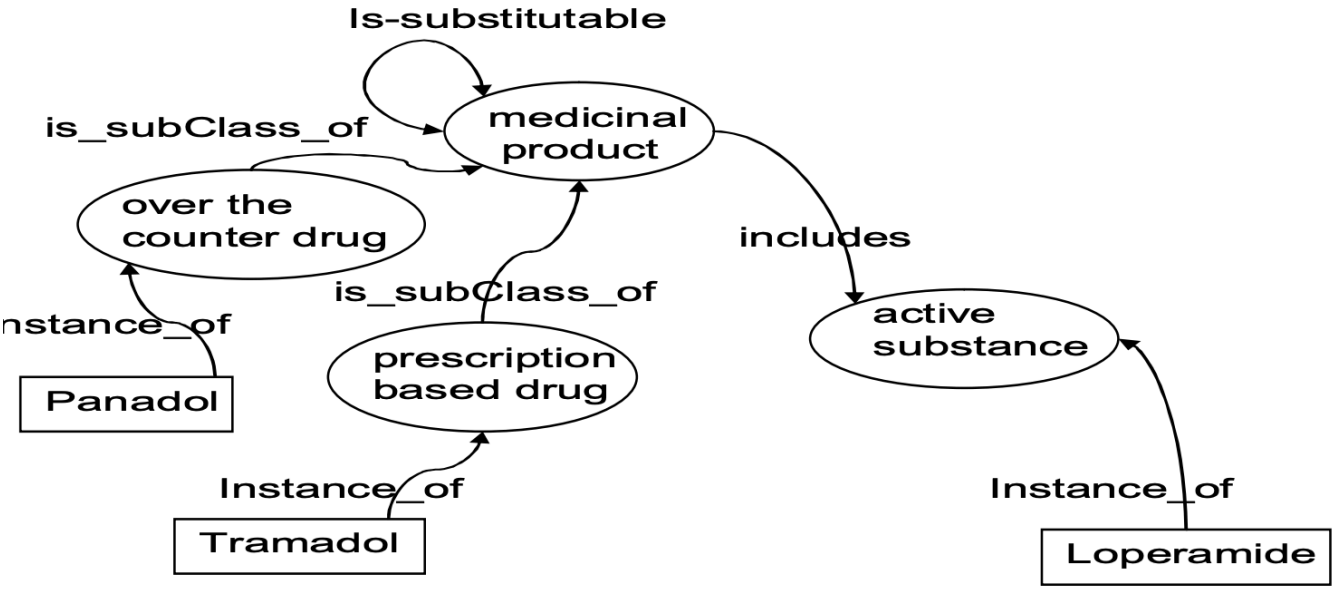
▶ Đáp án

NNF a) `¬Assistant ⊔ ∀teaches.LaboratoryCourse`. Phản chứng: `Assistant(i)`, `∃teaches.Course(i)`, `∀teaches.¬LaboratoryCourse(i)`. Luật 6: `teaches(i,b)`, `Course(b)`. Luật 7 (từ phản chứng): `¬LaboratoryCourse(b)`. 3a(i) → 5: `¬Assistant(i)` ✗ | `∀teaches.LaboratoryCourse(i)` → 7: `LaboratoryCourse(b)` ✗. Mọi nhánh đóng ⇒ **đúng**.

V6. Viết SPARQL trên ontology này → xem trang D8 (Q1–Q10 chạy trên dữ liệu `ex3-lab-course`).

■ Lời giải Part 3 Ex2 (thuốc)

> ***Đề:*** *Medicinal products include over-the-counter drugs and prescription based drugs. Each drug consists of an active substance. In addition, each drug can be replaced by zero or more other drugs. Panadol is an OTC drug. Tramadol is a prescription based drug. Loperamide is an active substance.*



Câu	Tiên đề	Ghi chú
include OTC & prescription drugs	`OTCDrug ⊑ MedicinalProduct`, `PrescriptionDrug ⊑ MedicinalProduct`, `OTCDrug ⊓ PrescriptionDrug ⊑ ⊥`	covering (`MedicinalProduct ≡ OTC ⊔ Prescription`) chỉ thêm nếu hiểu "include" là "gồm đúng"
Each drug consists of an active substance	`MedicinalProduct ⊑ ∃includes.ActiveSubstance`	"an" = ít nhất một; nếu hiểu "đúng một" thì `=1 includes.ActiveSubstance`
can be replaced by **zero or more** other drugs	`isSubstitutableBy`: domain/range MedicinalProduct; *(tùy chọn)* Symmetric, Irreflexive ("other")	**zero or more ⇒ không cần restriction** (`min 0` luôn đúng) — câu bẫy
Panadol / Tramadol / Loperamide	`OTCDrug(Panadol)`, `PrescriptionDrug(Tramadol)`, `ActiveSubstance(Loperamide)`	
(thiết kế tốt)	`ActiveSubstance ⊓ MedicinalProduct ⊑ ⊥`	

```
Class: MedicinalProduct      SubClassOf: includes some ActiveSubstance
Class: OverTheCounterDrug    SubClassOf: MedicinalProduct
Class: PrescriptionBasedDrug SubClassOf: MedicinalProduct   DisjointWith: OverTheCounterDrug
Class: ActiveSubstance       DisjointWith: MedicinalProduct
ObjectProperty: includes           Domain: MedicinalProduct   Range: ActiveSubstance
ObjectProperty: isSubstitutableBy  Domain: MedicinalProduct   Range: MedicinalProduct
                                   Characteristics: Symmetric, Irreflexive
Individual: Panadol    Types: OverTheCounterDrug
Individual: Tramadol   Types: PrescriptionBasedDrug
Individual: Loperamide Types: ActiveSubstance
```

Suy diễn đáng nói: Panadol là MedicinalProduct (subClassOf) và **có một hoạt chất nào đó** (∃) — chưa biết là gì (OWA). Nếu thêm `Panadol isSubstitutableBy Tramadol` ⇒ `Tramadol isSubstitutableBy Panadol` (Symmetric). Nếu thêm `Loperamide a OverTheCounterDrug` ⇒ **inconsistent** (ActiveSubstance disjoint MedicinalProduct).

■ Lời giải Part 3 Ex4 (phim)

> ***Đề:*** *Class: Movie, Man, Woman, Person, LoveMovie, AnimalDocumentary, AnimatedMovie · Property: title, director, leading-actor, supporting-actor, actor, male-lead, female-lead.*

Mô tả	DL	Manchester
Any Man or Woman is a Person; Person can only be Man or Woman, not both	`Person ≡ Man ⊔ Woman`, `Man ⊓ Woman ⊑ ⊥`	`Class: Person DisjointUnionOf: Man, Woman`
leading actor và supporting actor đều là actor	`leading-actor ⊑ actor`, `supporting-actor ⊑ actor`	`SubPropertyOf: actor`
A male-lead is a Man	`⊤ ⊑ ∀male-lead.Man` (range)	`ObjectProperty: male-lead Range: Man`
A female-lead is a Woman	`⊤ ⊑ ∀female-lead.Woman`	`Range: Woman`
male/female lead là leading actor (vai nối phim → người)	`male-lead ⊑ leading-actor`, `female-lead ⊑ leading-actor`; `actor`: domain Movie, range Person	
A love movie has ≥1 male lead and ≥1 female lead	`LoveMovie ⊑ Movie ⊓ ∃male-lead.Man ⊓ ∃female-lead.Woman`	`LoveMovie SubClassOf: Movie, male-lead some Man, female-lead some Woman`

Mô tả	DL	Manchester
A film without an actor is AnimalDocumentary or AnimatedMovie	`Movie ⊓ ¬∃actor.⊤ ⊑ AnimalDocumentary ⊔ AnimatedMovie` (= `Movie ⊓ =0 actor`)	GCI: `Movie and (actor max 0)` SubClassOf `AnimalDocumentary or AnimatedMovie`

Turtle đầy đủ: dữ liệu `ex4-film`. Điểm cú pháp: câu cuối là **GCI** (vế trái là biểu thức phức) ⇒ trong Turtle chủ ngữ là một blank node `[ owl:intersectionOf ( :Movie [ … owl:cardinality 0 ] ) ] rdfs:subClassOf [ owl:unionOf ( … ) ]`.

Các nhận xét ăn điểm:

1. **OWA với câu cuối:** phim "Our Planet" trong dữ liệu **không có triple actor** nhưng reasoner **không** xếp nó vào AnimalDocumentary ⊔ AnimatedMovie — vì "không ghi actor" ≠ "không có actor". Phải khẳng định đóng: `film:planet a [ a owl:Restriction ; owl:onProperty :actor ; owl:cardinality 0 ]`.
2. **LoveMovie: SubClassOf hay EquivalentTo?** Đề nói "has" (điều kiện cần) ⇒ SubClassOf. Nếu dùng EquivalentTo ⇒ Titanic (`male-lead` Leo, `female-lead` Kate) **tự được xếp** là LoveMovie.
3. **Suy diễn từ dữ liệu Avatar:** `people:sam` là leading-actor ⇒ là actor (subPropertyOf) ⇒ là Person (range) ✓. Nhưng **không** suy ra được `male-lead` từ `leading-actor` + `Man` (OWL không có luật "nếu–thì" kiểu này) ⇒ cần SWRL, hoặc mẹo OWL 2: `Man ≡ ∃isMan.Self`, `leading-actor ∘ isMan ⊑ male-lead`.
4. `director` nên có range Person (James là Man ⇒ Person). `title` là DatatypeProperty (dùng `dc:title` của Dublin Core cho tái sử dụng).

---

④ Đề thi cũ dạng này

■ 2023-3 · Câu 2 — RDFS + OWL + DL (phương tiện giao thông)

> ***Dạng:*** *D5* · ***Nguồn:*** *2023-C2*

Định nghĩa các lớp và các thuộc tính dựa trên RDF, RDFS và OWL sử dụng cú pháp Turtle. Các lớp: Phương tiện giao thông, phương tiện đường bộ, phương tiện đường thủy, phương tiện đường không, đường giao thông, phương tiện lưỡng cư. Các thuộc tính: Vận hành, nhiên liệu. *(Được tùy ý bổ sung các lớp và các thuộc tính cần thiết.)*

a) Phương tiện giao thông đường bộ, phương tiện giao thông đường thủy, và phương tiện giao thông đường hàng không là các loại phương tiện giao thông. Đường giao thông là tên gọi chung cho đường bộ, đường thủy, đường hàng không. b) Thuộc tính vận hành có miền là phương tiện giao thông và khoảng là đường giao thông. Thuộc tính nhiên liệu có miền là phương tiện giao thông, và giá trị có thể là xăng, dầu, hoặc điện. c) Phương tiện lưỡng cư là phương tiện thuộc 2 lớp nhưng không bắt buộc phải đúng 2 lớp. *(Ví dụ: dựa trên các dữ kiện Amphicar vừa là Phương tiện đường bộ vừa là Phương tiện đường thủy có thể suy ra nó là Phương tiện lưỡng cư, không phụ thuộc vào việc nó có thể bay hay không.)* d) Biểu diễn lớp Phương tiện lưỡng cư, các thuộc tính vận hành, nhiên liệu bằng các biểu thức DL.

◇ Gợi ý / Khuôn

Câu trong đề	Cấu trúc
"X, Y, Z là **các loại** W"	`X rdfs:subClassOf W` (mỗi lớp một triple)
"W là **tên gọi chung** cho X, Y, Z"	`W owl:equivalentClass [ owl:unionOf (X Y Z) ]` (DL: W ≡ X ⊔ Y ⊔ Z)
"miền / khoảng"	`rdfs:domain` / `rdfs:range` (DL: ∃p.⊤ ⊑ D, ⊤ ⊑ ∀p.R)
"giá trị **có thể là** a, b, c"	range là lớp liệt kê `[ owl:oneOf (:a :b :c) ]`
"thuộc **(ít nhất) 2 trong 3 lớp**"	**hợp của các giao từng cặp**: (A⊓B) ⊔ (A⊓C) ⊔ (B⊓C)

Phải dùng ≡ (owl:equivalentClass, hoặc unionOf đặt trực tiếp như lời giải của thầy) chứ không dùng ⊑. Lý do: đề yêu cầu **suy ra được** Amphicar là lưỡng cư, mà muốn suy theo chiều "đủ điều kiện ⇒ thuộc lớp" thì định nghĩa phải là tương đương.

▶ Đáp án

```
@prefix : <http://semweb.edu.vn/exam#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
# a)
:Phương_Tiện_Đường_Bộ rdfs:subClassOf :Phương_Tiện_Giao_Thông .
:Phương_Tiện_Đường_Thủy rdfs:subClassOf :Phương_Tiện_Giao_Thông .
:Phương_Tiện_Đường_Không rdfs:subClassOf :Phương_Tiện_Giao_Thông .
:Đường_Giao_Thông owl:unionOf (:Đường_Bộ :Đường_Thủy :Đường_Không) .
# b)
:vận_hành rdfs:domain :Phương_Tiện_Giao_Thông ; rdfs:range :Đường_Giao_Thông .
:nhiên_liệu rdfs:domain :Phương_Tiện_Giao_Thông ;
    rdfs:range [ owl:oneOf (:xăng :dầu :điện) ] .
# c)
:Phương_Tiện_Lưỡng_Cư owl:unionOf (
    [ owl:intersectionOf (:Phương_Tiện_Đường_Bộ :Phương_Tiện_Đường_Thủy) ]
    [ owl:intersectionOf (:Phương_Tiện_Đường_Bộ :Phương_Tiện_Đường_Không) ]
    [ owl:intersectionOf (:Phương_Tiện_Đường_Thủy :Phương_Tiện_Đường_Không) ] ) .
```

Muốn viết chuẩn OWL 2 DL hơn thì khai báo kiểu và dùng `owl:equivalentClass`. Hai cách đều được chấp nhận:

```
:Phương_Tiện_Lưỡng_Cư a owl:Class ; owl:equivalentClass [ a owl:Class ; owl:unionOf ( … như trên
… ) ] .
:vận_hành a owl:ObjectProperty .  :nhiên_liệu a owl:ObjectProperty .
:xăng a :Nhiên_Liệu . :dầu a :Nhiên_Liệu . :điện a :Nhiên_Liệu .
```

d) Biểu thức DL

```
Phương_Tiện_Lưỡng_Cư ≡ (Phương_Tiện_Đường_Bộ ⊓ Phương_Tiện_Đường_Thủy)
                     ⊔ (Phương_Tiện_Đường_Bộ ⊓ Phương_Tiện_Đường_Không)
                     ⊔ (Phương_Tiện_Đường_Thủy ⊓ Phương_Tiện_Đường_Không)
∃vận_hành.⊤ ⊑ Phương_Tiện_Giao_Thông        (miền)
⊤ ⊑ ∀vận_hành.Đường_Giao_Thông               (khoảng)
∃nhiên_liệu.⊤ ⊑ Phương_Tiện_Giao_Thông      (miền)
⊤ ⊑ ∀nhiên_liệu.{xăng, dầu, điện}           (khoảng)
```

Lưu ý: bản lời giải tham chiếu gõ nhầm khoảng của vận_hành thành `∀vận_hành.Phương_Tiện_Giao_Thông`. Khoảng đúng là **Đường_Giao_Thông**, như ghi ở trên.

Kiểm tra với ví dụ trong đề: Amphicar ∈ Bộ ⊓ Thủy, nên Amphicar thuộc vế phải của hợp, và vì có ≡ nên suy ra Amphicar ∈ Lưỡng_Cư. Kết luận này không phụ thuộc Amphicar có bay được hay không.

■ 2022-1 · Câu 4 — DL + RDFS/OWL (thú cưng)

Dạng: *D5* · ***Nguồn:*** *2022-C4*

Biểu diễn các khái niệm theo mô tả sau: *Thú cưng gồm có chó và mèo và không có thú cưng nào vừa là chó và vừa là mèo.* a) Bằng DL. b) Bằng RDFS, OWL sử dụng Turtle.

◇ Gợi ý / Khuôn

- "**gồm có** chó và mèo" nghĩa là chó và mèo là lớp con của thú cưng (⊑). Nếu hiểu chặt hơn là "chỉ gồm" thì viết Pet ≡ Dog ⊔ Cat.
- "**không có** cái nào vừa là A vừa là B" nghĩa là A ⊓ B ⊑ ⊥, tức là `owl:disjointWith`.

▶ Đáp án

a) DL:

```
Dog ⊑ Pet
Cat ⊑ Pet
Dog ⊓ Cat ⊑ ⊥
```

b) Turtle:

```
@prefix bio: <http://exam.hust.edu.vn/bio/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
bio:Cat rdfs:subClassOf bio:Pet .
bio:Dog rdfs:subClassOf bio:Pet .
bio:Cat owl:disjointWith bio:Dog .
```

Có thể viết theo cách khác, ví dụ `bio:Pet owl:equivalentClass [ owl:unionOf (bio:Dog bio:Cat) ]`, tương ứng Pet ≡ Dog ⊔ Cat, nếu hiểu thú cưng **chỉ** gồm chó và mèo.

■ 6390 · Câu 2 — RDFS + OWL (phần mềm C++)

Dạng: *D5* · ***Nguồn:*** *6390-C2*

Hãy tạo biểu diễn cho các khái niệm sau bằng RDFS và OWL, sử dụng định dạng Turtle: a. Phụ thuộc (giữa các gói phần mềm) là 1 thuộc tính đối tượng có tính chất bắc cầu. b. Phụ thuộc tương đương với sử dụng. c. Trình biên dịch C++ gồm có g++, clang++, msvc. d. Chương trình C++ là những chương trình được viết bằng ngôn ngữ C++ và được biên dịch bằng trình biên dịch C++. e. Chương trình đa nền tảng là chương trình có thể hoạt động tối thiểu trên 2 hệ điều hành khác nhau.

◇ Gợi ý / Khuôn

Câu	Cấu trúc OWL	DL
bắc cầu	`owl:ObjectProperty, owl:TransitiveProperty` + domain/range	Trans(phụThuộc)
tương đương (thuộc tính)	`owl:equivalentProperty`	phụThuộc ≡ sửDụng
"gồm có a, b, c" (cá thể cụ thể)	`owl:oneOf (…)` hoặc khai báo từng cá thể	TBD ≡ {a, b, c}
"là những X **được** … **và được** …"	`owl:equivalentClass [ owl:intersectionOf ( X restriction1 restriction2 ) ]`	X ⊓ ∃p.{CPP} ⊓ ∃q.TBD
"tối thiểu 2 Y"	`owl:minQualifiedCardinality "2"^^xsd:nonNegativeInteger ; owl:onClass Y`	≥2 p.Y

▶ Đáp án

```
@prefix : <http://semweb.edu.vn/exam#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

# Khai báo các lớp và thuộc tính được dùng bên dưới
:GoiPhanMem a owl:Class .
:ChuongTrinh a owl:Class .
:HeDieuHanh a owl:Class .
:NgonNguLapTrinh a owl:Class .
:vietBang a owl:ObjectProperty .
:bienDichBoi a owl:ObjectProperty .
:hoatDongTren a owl:ObjectProperty .

# a. Phụ thuộc là thuộc tính đối tượng bắc cầu giữa các gói phần mềm
:phuThuoc a owl:ObjectProperty , owl:TransitiveProperty ;
    rdfs:domain :GoiPhanMem ; rdfs:range :GoiPhanMem .

# b. Phụ thuộc tương đương với sử dụng
:suDung a owl:ObjectProperty .
:phuThuoc owl:equivalentProperty :suDung .

# c. Trình biên dịch C++ gồm có g++, clang++, msvc
:TrinhBienDichCPP a owl:Class ;
    owl:equivalentClass [ a owl:Class ; owl:oneOf (:gpp :clangpp :msvc) ] .
:gpp a :TrinhBienDichCPP ; rdfs:label "g++" .
:clangpp a :TrinhBienDichCPP ; rdfs:label "clang++" .
:msvc a :TrinhBienDichCPP ; rdfs:label "msvc" .

# d. Chương trình C++ = chương trình viết bằng C++ VÀ biên dịch bằng trình biên dịch C++
:CPP a :NgonNguLapTrinh .
:ChuongTrinhCPP a owl:Class ;
    owl:equivalentClass [ a owl:Class ;
        owl:intersectionOf ( :ChuongTrinh
            [ a owl:Restriction ; owl:onProperty :vietBang ; owl:hasValue :CPP ]
            [ a owl:Restriction ; owl:onProperty :bienDichBoi ; owl:someValuesFrom
:TrinhBienDichCPP ] ) ] .

# e. Chương trình đa nền tảng: hoạt động trên tối thiểu 2 hệ điều hành
:ChuongTrinhDaNenTang a owl:Class ;
    owl:equivalentClass [ a owl:Class ;
        owl:intersectionOf ( :ChuongTrinh
            [ a owl:Restriction ; owl:onProperty :hoatDongTren ;
              owl:minQualifiedCardinality "2"^^xsd:nonNegativeInteger ;
              owl:onClass :HeDieuHanh ] ) ] .
```

Biểu thức DL tương ứng:

```
a. Trans(phuThuoc);  ∃phuThuoc.⊤ ⊑ GoiPhanMem;  ⊤ ⊑ ∀phuThuoc.GoiPhanMem
b. phuThuoc ≡ suDung
c. TrinhBienDichCPP ≡ {gpp, clangpp, msvc}
d. ChuongTrinhCPP ≡ ChuongTrinh ⊓ ∃vietBang.{CPP} ⊓ ∃bienDichBoi.TrinhBienDichCPP
e. ChuongTrinhDaNenTang ≡ ChuongTrinh ⊓ ≥2 hoatDongTren.HeDieuHanh
```

Ghi chú ăn điểm:

- (c) Nếu chỉ hiểu "gồm có" là "có các thành viên này", không bắt buộc là "chỉ gồm", thì viết 3 triple `a :TrinhBienDichCPP` là đủ. `owl:oneOf` thì đóng lớp, tức là chỉ có đúng 3 trình biên dịch.
- (e) Cụm "khác nhau": OWL **không có giả thiết tên duy nhất** (UNA). Muốn reasoner đếm được 2 hệ điều hành là 2 cá thể khác nhau thì phải khai báo thêm, ví dụ `[] a owl:AllDifferent ; owl:distinctMembers (:windows`

`:linux :macos`).

- (d) Dùng `owl:equivalentClass` chứ không dùng subClassOf, vì đề nói "là **những** chương trình …", tức là một định nghĩa có điều kiện đủ.

D6 Suy diễn OWL / reasoner

Đề hỏi reasoner suy ra được gì, hoặc cần thêm tiên đề OWL nào để suy ra X (và không suy ra Y).

③ Bài tập tuần: Pizza (P3 Ex1)

> *Số liệu bên dưới đọc trực tiếp từ pizza ontology (lớp tên `Margherita`, không phải `MargheritaPizza` như trong tutorial).*

1. Xem định nghĩa class/property — 99 class, 8 object property, 5 cá thể (America, England, France, Germany, Italy — `AllDifferent`).

2. Margherita có phải VegetarianPizza không? Vì sao?

- `Margherita ⊑ NamedPizza ⊓ ∃hasTopping.MozzarellaTopping ⊓ ∃hasTopping.TomatoTopping ⊓ ∀hasTopping.(MozzarellaTopping ⊔ TomatoTopping)` ← **closure axiom** (dòng `only`).
- `VegetarianPizza ≡ Pizza ⊓ ¬∃hasTopping.MeatTopping ⊓ ¬∃hasTopping.FishTopping`.
- Mozzarella ⊑ CheeseTopping, Tomato ⊑ VegetableTopping; `AllDisjointClasses(CheeseTopping, FishTopping, …, MeatTopping, …, VegetableTopping)` ⇒ mọi topping của Margherita không thể là Meat/Fish ⇒ **CÓ, Margherita ⊑ VegetarianPizza** (suy ra, không khai báo).
- **Vì sao cần closure axiom:** bỏ dòng `only` đi thì theo **OWA** Margherita vẫn có thể có topping thịt chưa được nhắc tới ⇒ reasoner **không** xếp vào VegetarianPizza (và cũng **không** xếp vào NonVegetarianPizza — không chứng minh được cả hai chiều).

3. hasIngredient: domain `Food`, range `Food`; **subproperties** `hasBase`, `hasTopping`; **inverse** `isIngredientOf`; đặc tính **Transitive** (hasTopping thêm InverseFunctional, domain Pizza, range PizzaTopping).

4. Classify — asserted vs inferred hierarchy; owl:Nothing.

- **Asserted** = cây do người viết khai báo; **inferred** = cây reasoner tính ra (thêm subclass suy được từ định nghĩa, đa kế thừa). Khuyến nghị: asserted là cây đơn giản, để reasoner tính đa kế thừa ("normalisation").
- **Lớp là subclass của owl:Nothing** = lớp **không thỏa được** (unsatisfiable): không thể có cá thể nào mà không gây mâu thuẫn. (Khác với *ontology inconsistent* — khi một cá thể bị khẳng định thuộc lớp như vậy.)
- Hai lớp trong pizza ontology:
  - **CheeseyVegetableTopping** ⊑ CheeseTopping ⊓ VegetableTopping, mà hai lớp này **disjoint** ⇒ rỗng. (Tutorial gọi là ProbeInconsistentTopping.)
  - **IceCream** ⊑ `∃hasTopping.FruitTopping`; `hasTopping` có **domain Pizza** ⇒ IceCream ⊑ Pizza; nhưng `AllDisjointClasses(IceCream, Pizza, PizzaBase, PizzaTopping)` ⇒ rỗng. **Bài học: domain là tiên đề suy diễn, không phải ràng buộc kiểm tra** — dùng `hasTopping` cho kem làm kem "trở thành" pizza.
- Siêu lớp suy ra của Margherita: **CheeseyPizza** (`≡ Pizza ⊓ ∃hasTopping.CheeseTopping`), **VegetarianPizza** (và các lớp tương đương VegetarianPizzaEquivalent1/2). *Không* phải InterestingPizza (`≥3 hasTopping` — chỉ biết 2).

5. Grandiosa ⊑ NamedPizza ⊓ ∃hasTopping.HamTopping ⊓ ∃hasTopping.TomatoTopping ⊓ ∃hasTopping.CheeseTopping. Siêu lớp suy ra:

Siêu lớp	Lý do
CheeseyPizza	Pizza ⊓ ∃hasTopping.CheeseTopping
MeatyPizza	HamTopping ⊑ MeatTopping ⇒ ∃hasTopping.MeatTopping
NonVegetarianPizza	có topping thịt ⇒ ¬VegetarianPizza ⇒ `Pizza ⊓ ¬VegetarianPizza`
InterestingPizza	3 topping thuộc 3 lớp **đôi một disjoint** (Meat, Vegetable, Cheese) ⇒ chắc chắn là 3 cá thể khác nhau ⇒ `≥3 hasTopping`

6. Grandiosa đến từ Norway; Norway khác các nước đã có.

```
# fragment
:Grandiosa rdfs:subClassOf [ a owl:Restriction ; owl:onProperty :hasCountryOfOrigin ; owl:hasValue
:Norway ] .
:Norway a :Country , owl:NamedIndividual .
[] a owl:AllDifferent ; owl:distinctMembers ( :America :England :France :Germany :Italy :Norway )
.
```

- Trong pizza ontology: **`Country ≡ DomainConcept ⊓ {America, England, France, Germany, Italy}`** — **lớp đóng** (oneOf).
- ⇒ Norway là Country thì phải `sameAs` một trong 5 nước; nhưng ta khai báo Norway khác cả 5 ⇒ **ontology INCONSISTENT** (đúng ví dụ "Mike in PeopleInD219" trên slide 05).
- Cách sửa: thêm Norway vào danh sách `oneOf` của Country, **hoặc** không khai báo `:Norway a :Country` (khi đó `hasCountryOfOrigin` không có range ⇒ không suy ra Norway là Country ⇒ consistent).
- Thêm nữa: Grandiosa **không** được suy ra là `RealItalianPizza` (≡ Pizza ⊓ ∃hasCountryOfOrigin.{Italy}), nhưng cũng **không** suy ra được là *không phải* — `hasCountryOfOrigin` không Functional nên pizza có thể có nhiều nước xuất xứ (OWA).

---

④ Đề thi cũ dạng này

■ 2022-1 · Câu 3 — RDFS hay OWL? (Molly là mèo, Spot thì không)

> ***Dạng:*** *D6* · ***Nguồn:*** *2022-C3*

```
@prefix bio: <http://exam.hust.edu.vn/bio/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
bio:offspring rdfs:domain bio:Animal; rdfs:range bio:Animal .
bio:Fido a bio:Dog; bio:offspring bio:Spot.
bio:Milo a bio:Cat; bio:offspring bio:Molly.
```

Trong tầng ngữ nghĩa nào (RDFS hay OWL) có thể bổ sung các ràng buộc để có thể suy diễn Molly là mèo, nhưng không suy ra Spot là mèo? Hãy bổ sung các ràng buộc đó.

◇ Gợi ý / Khuôn

- Trong RDFS, khoảng (range) của một thuộc tính là **toàn cục**: mọi giá trị của thuộc tính đều thuộc lớp đó, không phân biệt chủ thể là ai. Nếu đặt range là Cat thì Spot cũng thành mèo. Vậy **RDFS không làm được**.
- OWL có **restriction gắn vào lớp**, ví dụ "con của mèo là mèo". Có hai cách làm:
  1. dùng thuộc tính ngược `sprungBy` kèm `allValuesFrom`;
  2. viết `Cat ⊑ ∀offspring.Cat`.

▶ Đáp án

Lựa chọn: OWL.

Với CSTT đã cho, trong phạm vi RDFS chỉ suy ra được Molly và Spot là động vật. Nếu đổi khoảng của offspring thành Cat thì suy ra **cả** Molly và Spot là mèo. RDFS không cho phép ràng buộc khoảng theo miền, nên không biểu diễn được yêu cầu đề bài.

OWL cho phép định nghĩa lớp dựa trên giới hạn giá trị thuộc tính. Lời giải tham chiếu như sau:

```
@prefix bio: <http://exam.hust.edu.vn/bio/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
bio:sprungBy owl:inverseOf bio:offspring .
bio:Cat owl:equivalentClass [ a owl:Restriction ;
    owl:onProperty bio:sprungBy ; owl:allValuesFrom bio:Cat ] .
```

Cách khác ngắn hơn, cũng đúng:

```
bio:Cat rdfs:subClassOf [ a owl:Restriction ; owl:onProperty bio:offspring ; owl:allValuesFrom
bio:Cat ] .
```

Viết bằng DL: Cat ⊑ ∀offspring.Cat. Milo ∈ Cat và (Milo, Molly) ∈ offspring nên Molly ∈ Cat. Không có dữ kiện Fido là Cat (Fido là Dog), nên không suy ra gì về Spot.

Lưu ý về cách của thầy (Cat ≡ ∀sprungBy.Cat): chiều ⊒ cho rằng nếu mọi cha/mẹ đều là mèo thì là mèo. Dưới giả thiết thế giới mở, không thể biết Molly chỉ có duy nhất Milo là cha/mẹ, nên chiều này **không tự suy ra** Molly là mèo. Vì vậy khi đi thi nên ghi **cả hai cách**, hoặc dùng cách ∀offspring cho chắc.

■ 6390 · Câu 3 — Suy diễn OWL với allValuesFrom

Dạng: *D6* · ***Nguồn:*** *6390-C3*

```
@prefix : <http://semweb.edu.vn/exam#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
:C rdfs:subClassOf :D, :E;
   owl:equivalentClass [a owl:Restriction; owl:onProperty :p; owl:allValuesFrom :F].
:x :p :e, :f.   :y :p :f.   :e a :E.   :f a :F.
```

a) Tìm 1 bộ-3 liên quan tới :C có thể được suy ra từ CSTT đã cho đồng thời giải thích cách suy diễn. b) Giải thích ngữ nghĩa hình thức của lớp :C bằng cách chuyển các bộ-3 về :C sang lô-gic mô tả.

◇ Gợi ý / Khuôn

- Đây là **bẫy giả thiết thế giới mở (OWA)**. :y chỉ có một giá trị p là :f, và :f ∈ F. Dễ nghĩ ngay rằng ":y a :C", nhưng điều đó **SAI**: dữ liệu không nói :y không có giá trị p nào khác, nên không thể kết luận "mọi giá trị p của :y đều thuộc F".
- Triple **chắc chắn** suy ra được nằm ở tầng lớp:
  - equivalentClass kéo theo subClassOf theo cả hai chiều;
  - kết hợp với C ⊑ D và C ⊑ E bằng tính bắc cầu, ta được "lớp restriction ⊑ D, E".

▶ Đáp án

a) Gọi `_:r` là lớp restriction `[a owl:Restriction; owl:onProperty :p; owl:allValuesFrom :F]`.

```
(1) :C rdfs:subClassOf _:r .      <=  :C owl:equivalentClass _:r .        (eqc ⇒ sco, chiều 1)
(2) _:r rdfs:subClassOf :C .      <=  :C owl:equivalentClass _:r .        (eqc ⇒ sco, chiều 2)
(3) _:r rdfs:subClassOf :D .      <=  (2)  và  :C rdfs:subClassOf :D .    (bắc cầu rdfs11)
    _:r rdfs:subClassOf :E .      <=  (2)  và  :C rdfs:subClassOf :E .    (bắc cầu rdfs11)
```

Câu trả lời chính nên chọn **(3)**: mọi đối tượng có tất cả giá trị p thuộc F thì thuộc D (và E). Nói cách khác, ∀p.F ⊑ D ⊓ E.

Giải thích vì sao **không** suy ra được `:y a :C` và `:x a :C`:

- Với :y: OWL theo giả thiết **thế giới mở**. Biết (y, f) ∈ p và f ∈ F, nhưng không biết :y còn giá trị p nào khác không, nên không kết luận được y ∈ ∀p.F.
- Với :x: x có giá trị p là :e ∈ E, và không biết e ∈ F hay không. Vì vậy cũng **không** suy ra được `:x a :C`, mà cũng không suy ra được x ∉ C.
- Muốn suy ra được `:y a :C` thì phải "đóng" thông tin, ví dụ thêm `:y a [a owl:Restriction; owl:onProperty :p; owl:maxCardinality 1]`. Khi đó :f là giá trị p duy nhất của :y, nên y ∈ ∀p.F, rồi suy tiếp `:y a :C`, `:y a :D`, `:y a :E`.

b) Chuyển sang DL:

```
C ⊑ D
C ⊑ E
C ≡ ∀p.F
```

Ngữ nghĩa hình thức theo một diễn giải $I = (\Delta^I, \cdot^I)$:

$$C^I \subseteq D^I \;,\quad C^I \subseteq E^I$$
$$C^I = (\forall p.F)^I = \{\, a \in \Delta^I \mid \forall b : (a, b) \in p^I \rightarrow b \in F^I \,\}$$

Như vậy :C là tập các cá thể mà **mọi** giá trị p của nó (nếu có) đều thuộc F, và tập đó nằm trong $D \cap E$.

Lưu ý ngữ nghĩa: một cá thể **không có giá trị p nào** vẫn thuộc ∀p.F (thỏa mãn một cách hiển nhiên), nên cũng thuộc C.

D12 Tableau: NNF, 7 luật, consistency / subsumption

Đề bắt chứng minh bằng tableau: KB nhất quán không, a có thuộc C không, C có bao hàm trong D không.

③ Bài tập tuần (Pingu, Mary, Professor)

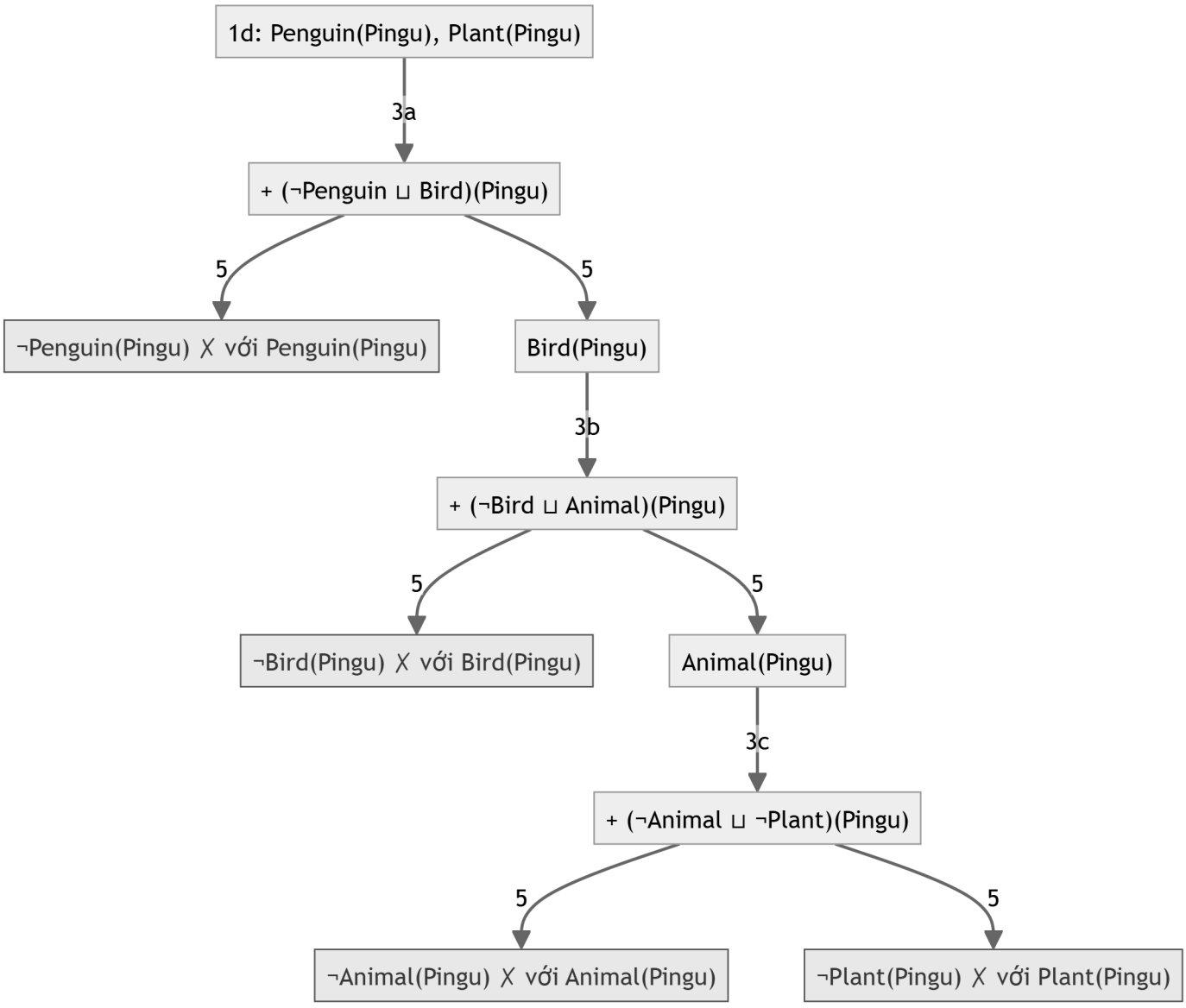
■ 3. Lời giải bài tập trên lớp

Ex1 — Consistency checking (Pingu)

Đề: `:Penguin rdfs:subClassOf :Bird . :Bird rdfs:subClassOf :Animal . :Animal owl:disjointWith :Plant . :Pingu a :Penguin , :Plant .`

DL → NNF

- a) `Penguin ⊑ Bird` ⟶ `¬Penguin ⊔ Bird`
- b) `Bird ⊑ Animal` ⟶ `¬Bird ⊔ Animal`
- c) `Animal ⊓ Plant ⊑ ⊥` ⟶ `¬Animal ⊔ ¬Plant`
- d) `Penguin(Pingu)`, `Plant(Pingu)`



Kết luận: mọi nhánh đóng ⇒ ontology **INCONSISTENT** (Pingu vừa là Animal — qua Penguin ⊑ Bird ⊑ Animal — vừa là Plant, mà Animal disjoint Plant).

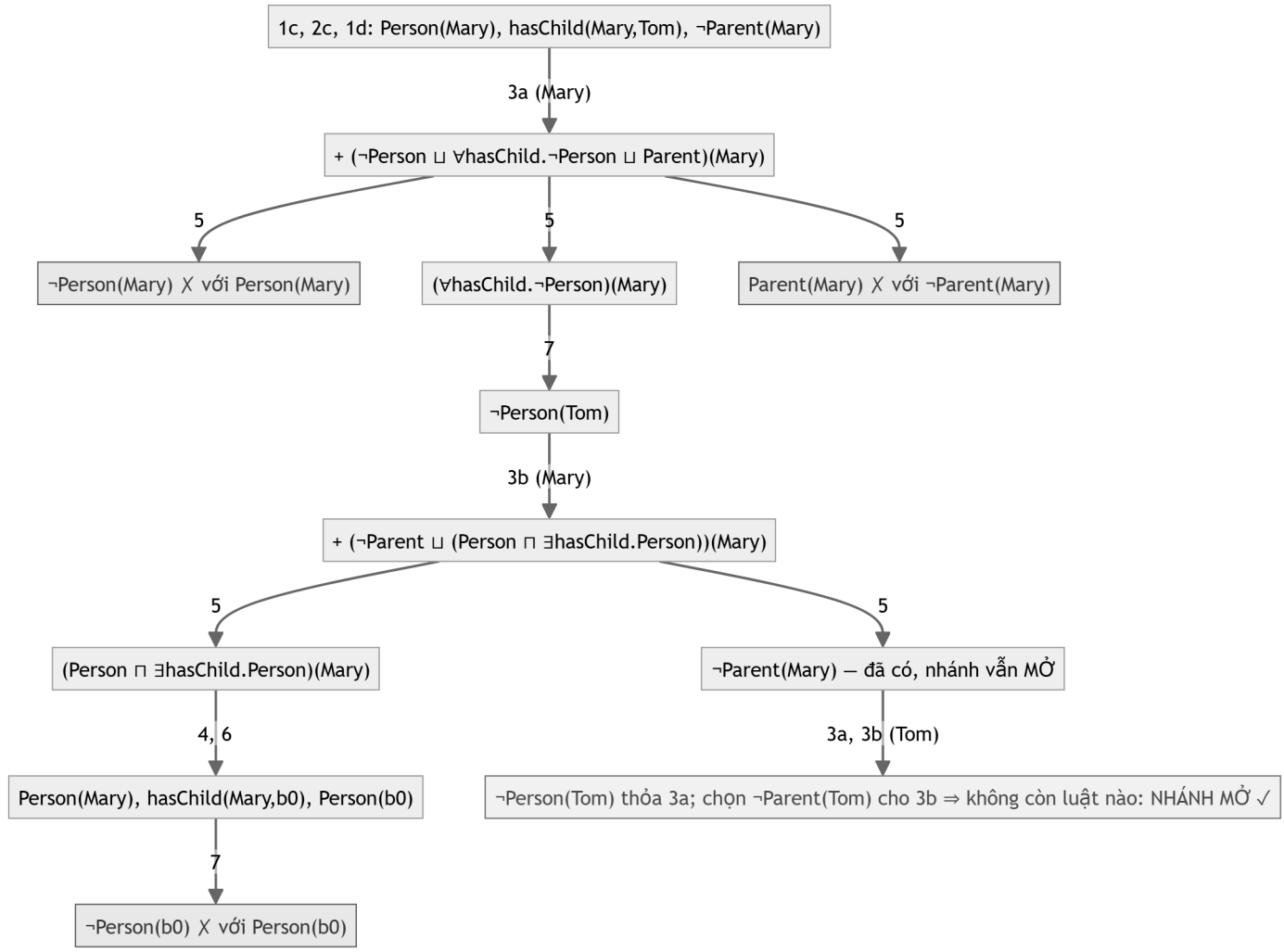
Ex2 — Class instantiation (Mary)

Đề: `Parent ≡ Person ⊓ ∃hasChild.Person`, `Person(Mary)`, `hasChild(Mary, Tom)`. (a) Suy ra được Mary là Parent không? (b) Thêm `Tom a :Person` thì sao?

Tách ≡ và NNF

- (1) `Person ⊓ ∃hasChild.Person ⊑ Parent` ⟶ a) `¬Person ⊔ ∀hasChild.¬Person ⊔ Parent`
- (2) `Parent ⊑ Person ⊓ ∃hasChild.Person` ⟶ b) `¬Parent ⊔ (Person ⊓ ∃hasChild.Person)`
- c) `Person(Mary)`, `hasChild(Mary, Tom)`
- d) **giả thiết phản chứng** `¬Parent(Mary)`

(a)



(Nhánh T1/T3 vẽ chung một lần tách 3 hướng = hai lần áp luật 5 liên tiếp.)

Kết luận (a): còn nhánh mở (mô hình: Mary là Person, Tom **không** là Person, Mary không là Parent — không mâu thuẫn) ⇒ **không suy ra được** `Parent(Mary)`. Lý do trực giác: OWA — không ai nói Tom là Person.

(b) Thêm `Person(Tom)`: ở nhánh duy nhất còn mở, luật 7 cho `¬Person(Tom)` ⇒ **✗ mâu thuẫn với Person(Tom)**. Hai nhánh kia vẫn đóng như trước ⇒ **mọi nhánh đóng** ⇒ thêm `¬Parent(Mary)` gây inconsistent ⇒ **Parent(Mary) được suy ra**.

Ex3 — Subsumption với lượng từ

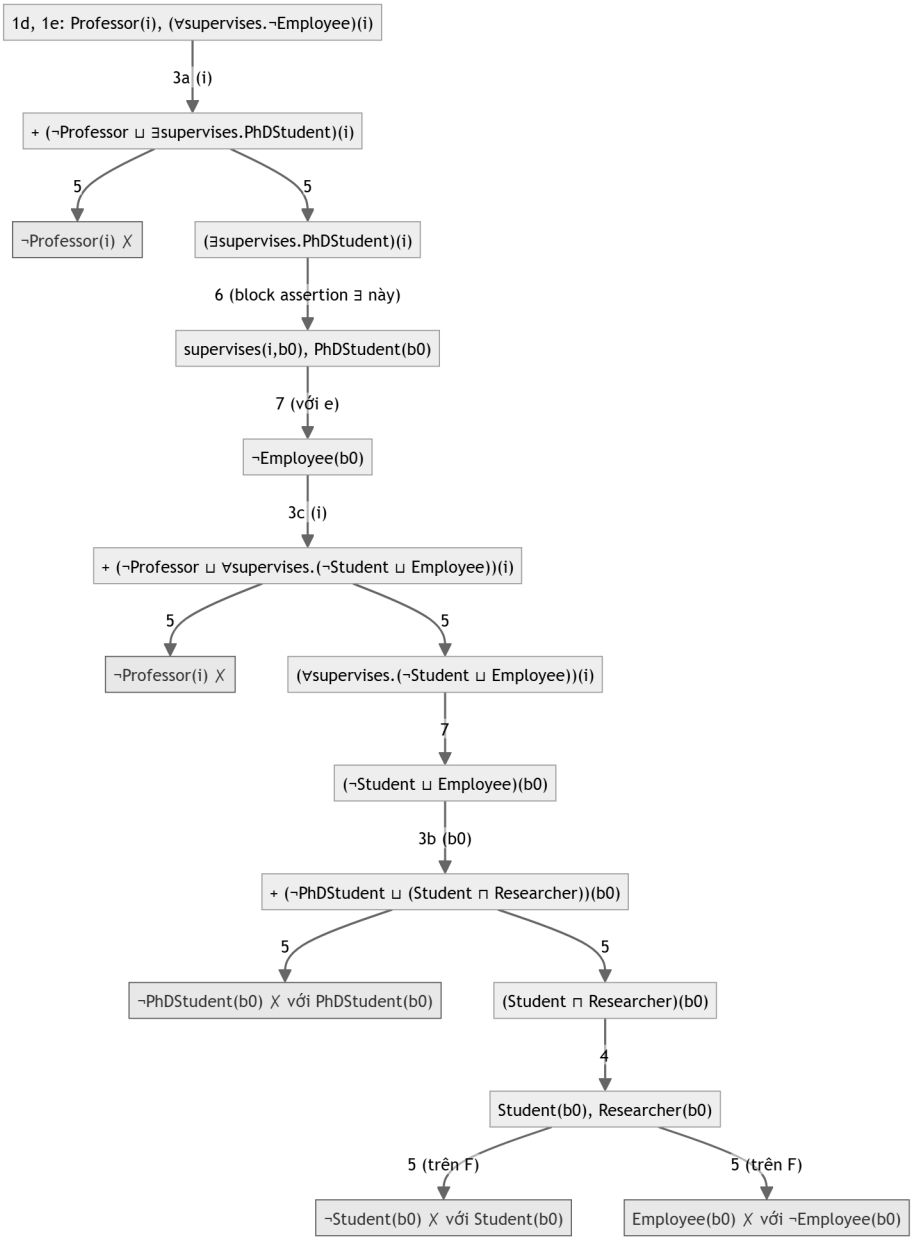
Đề: 1. `Professor ⊑ ∃supervises.PhDStudent` 2. `PhDStudent ⊑ Student ⊓ Researcher` 3. `Professor ⊑ ∀supervises.(¬Student ⊔ Employee)`. (a) Chứng minh hoặc bác bỏ `Professor ⊑ ∃supervises.Employee`. (b) Bỏ axiom 3 thì sao?

NNF

- a) `¬Professor ⊔ ∃supervises.PhDStudent`

- b) ¬PhDStudent ⊔ (Student ⊓ Researcher)
- c) ¬Professor ⊔ ∀supervises.(¬Student ⊔ Employee)
- **Phản chứng** cho subsumption: cá thể mới `i` với d) `Professor(i)` và e) ¬(∃supervises.Employee)(i) = **(∀supervises.¬Employee)(i)** (NNF của ¬∃).

(a)



Kết luận (a): mọi nhánh đóng ⇒ `Professor ⊓ ¬∃supervises.Employee` không thỏa được ⇒ **`Professor ⊑ ∃supervises.Employee` ĐÚNG** (mọi giáo sư giám sát ít nhất một Employee — chính là nghiên cứu sinh b0 kia: là Student nên theo axiom 3 phải là Employee).

(b) Bỏ axiom 3: cây giống trên nhưng không có bước 3c. Nhánh sau bước 4 chứa `Professor(i)`, `∀supervises.¬Employee(i)`, `supervises(i,b0)`, `PhDStudent(b0)`, `¬Employee(b0)`, `Student(b0)`, `Researcher(b0)`. Áp 3a cho b0 → chọn `¬Professor(b0)`: **không còn mâu thuẫn, không còn luật áp được ⇒ nhánh mở** ⇒ consistent ⇒ **không suy ra được** `Professor ⊑ ∃supervises.Employee`. Axiom 3 chính là "cầu nối" Student → Employee; thiếu nó, theo OWA ta không biết PhDStudent có là Employee hay không.

■ 4. Ví dụ trên slide (nên tự làm lại trên giấy)

- **Seth (tr. 47–54):** `Animal ⊓ Human ⊑ ⊥`, `Animal ≡ Mammal ⊔ Bird ⊔ Fish ⊔ Insect ⊔ Reptile`, `Human(Seth)`, `Insect(Seth)`. Chú ý: phải áp **cả hai chiều** của ≡ — chiều `Insect ⊑ Animal` (từ `¬(Mammal ⊔ … ⊔ Reptile) ⊔`

`Animal` = (`¬Mammal ⊓ … ⊓ ¬Reptile`) `⊔ Animal`) buộc Seth là Animal ⇒ đụng `Human`. Kết luận: **inconsistent**.

- **Jane (tr. 55–64):** `¬Woman ⊔ ∃hasMother.Woman`, `Woman(Jane)` ⇒ luật 6 sinh b0, b1, b2… vô hạn ⇒ cần **rule blocking** ⇒ kết luận **consistent**.
- **Reasoning tasks (tr. 24–32):** Jimmy (Human disjoint Animal, Jimmy thuộc cả hai ⇒ thuộc owl:Nothing), Married Bachelor (lớp không thỏa được), Flipper (thêm `¬Dolphin(Flipper)` để kiểm tra instance).

D7 Datalog: cây chứng minh + điểm cố định

Đề cho chương trình Datalog → vẽ cây chứng minh và tính điểm cố định B0 → B1 → ...

③ Bài luyện

■ Luyện tập 1 — Tổ tiên

Dạng: *D7* · ***Nguồn:*** *bài tự luyện*

```
Cha-mẹ(an, bình)   Cha-mẹ(bình, chi)   Cha-mẹ(chi, dũng)
L1: Tổ-tiên(X, Y) ← Cha-mẹ(X, Y)
L2: Tổ-tiên(X, Z) ← Cha-mẹ(X, Y) ∧ Tổ-tiên(Y, Z)
```

a) Vẽ cây chứng minh Tổ-tiên(an, dũng). b) Tìm điểm cố định.

◇ Gợi ý / Khuôn

L2 là luật **tuyến tính**: điều kiện thứ nhất là Cha-mẹ (một dữ kiện gốc), không phải Tổ-tiên. Vì vậy mỗi bước chỉ kéo dài chuỗi thêm đúng 1 bậc.

▶ Đáp án

```
B0: {Cha-mẹ(an,bình), Cha-mẹ(bình,chi), Cha-mẹ(chi,dũng)}
B1: + {Tổ-tiên(an,bình), Tổ-tiên(bình,chi), Tổ-tiên(chi,dũng)}          (L1)
B2: + {Tổ-tiên(an,chi), Tổ-tiên(bình,dũng)}                             (L2)
B3: + {Tổ-tiên(an,dũng)}                                                (L2: Cha-mẹ(an,bình) ∧
Tổ-tiên(bình,dũng))
B4: + {}   → Điểm cố định thu được ở Bước 3.
```

```
Tổ-tiên(an, dũng)                  [L2]
├── Cha-mẹ(an, bình)               (dữ kiện)
└── Tổ-tiên(bình, dũng)            [L2]
    ├── Cha-mẹ(bình, chi)          (dữ kiện)
    └── Tổ-tiên(chi, dũng)         [L1]
        └── Cha-mẹ(chi, dũng)      (dữ kiện)
```

■ Luyện tập 2 — Chuyến bay khứ hồi

Dạng: *D7* · ***Nguồn:*** *bài tự luyện*

```
Chuyến(HAN, DAD)   Chuyến(DAD, SGN)   Khứ-hồi(SGN, PQC)
L1: Chuyến(X, Y) ← Khứ-hồi(X, Y)
L2: Chuyến(Y, X) ← Khứ-hồi(X, Y)
L3: Đến-được(X, Y) ← Chuyến(X, Y)
L4: Đến-được(X, Z) ← Đến-được(X, Y) ∧ Chuyến(Y, Z)
```

a) Cây chứng minh Đến-được(HAN, PQC). b) Điểm cố định.

▶ Đáp án

Viết tắt: H = HAN, D = DAD, S = SGN, P = PQC.

```
B0: {Chuyến(H,D), Chuyến(D,S), Khứ-hồi(S,P)}
B1: + {Chuyến(S,P), Chuyến(P,S), Đến-được(H,D), Đến-được(D,S)}
      (L1, L2 với Khứ-hồi(S,P);  L3 với Chuyến(H,D), Chuyến(D,S))
B2: + {Đến-được(S,P), Đến-được(P,S), Đến-được(H,S), Đến-được(D,P)}
      (L3 với 2 chuyến mới;  L4: Đến(H,D)∧Chuyến(D,S);  Đến(D,S)∧Chuyến(S,P))
B3: + {Đến-được(H,P), Đến-được(P,P), Đến-được(S,S)}
      (L4: Đến(H,S)∧Chuyến(S,P);  Đến(P,S)∧Chuyến(S,P);  Đến(S,P)∧Chuyến(P,S))
B4: + {}   → Điểm cố định thu được ở Bước 3.
```

```
Đến-được(H, P)                  [L4]
├── Đến-được(H, S)              [L4]
│   ├── Đến-được(H, D)          [L3]
│   │   └── Chuyến(H, D)        (dữ kiện)
│   └── Chuyến(D, S)            (dữ kiện)
└── Chuyến(S, P)                [L1]
    └── Khứ-hồi(S, P)           (dữ kiện)
```

■ Luyện tập 3 — Luật RDFS viết bằng Datalog

Dạng: *D7 + D4* · ***Nguồn:*** *bài tự luyện*

```
Rel(owns, anna, fido)   Domain(owns, Person)   Range(owns, Dog)
Sub(Dog, Mammal)   Sub(Mammal, Animal)
L1: Type(X, C) ← Rel(P, X, Y) ∧ Domain(P, C)        (rdfs2)
L2: Type(Y, C) ← Rel(P, X, Y) ∧ Range(P, C)         (rdfs3)
L3: Type(X, C) ← Type(X, B) ∧ Sub(B, C)             (rdfs9)
L4: Sub(A, C) ← Sub(A, B) ∧ Sub(B, C)               (rdfs11)
```

a) Cây chứng minh Type(fido, Animal). b) Điểm cố định.

▶ Đáp án

```
B0: {Rel(owns,anna,fido), Domain(owns,Person), Range(owns,Dog), Sub(Dog,Mammal),
Sub(Mammal,Animal)}
B1: + {Type(anna,Person), Type(fido,Dog), Sub(Dog,Animal)}          (L1, L2, L4)
B2: + {Type(fido,Mammal), Type(fido,Animal)}                       (L3 với Sub(Dog,Mammal),
Sub(Dog,Animal))
B3: + {}   → Điểm cố định thu được ở Bước 2.
```

```
Type(fido, Animal)              [L3]
├── Type(fido, Dog)             [L2]
│   ├── Rel(owns, anna, fido)   (dữ kiện)
│   └── Range(owns, Dog)        (dữ kiện)
└── Sub(Dog, Animal)            [L4]
    ├── Sub(Dog, Mammal)        (dữ kiện)
    └── Sub(Mammal, Animal)     (dữ kiện)
```

Có một cây khác cũng đúng: đi qua Type(fido, Mammal) rồi dùng L3 với Sub(Mammal, Animal).

④ Đề thi cũ dạng này

■ 2023-3 · Câu 3 — Datalog: cây chứng minh + điểm cố định

Dạng: *D7* · ***Nguồn:*** *2023-C3*

Cho chương trình Datalog:

```
Đường-một-chiều(A, B)
Đường-một-chiều(B, C)
Đường-hai-chiều(D, C)
Đường-hai-chiều(Y, X) ← Đường-hai-chiều(X, Y)
Có-đường-đi(X, Y) ← Đường-hai-chiều(X, Y)
Có-đường-đi(X, Y) ← Đường-một-chiều(X, Y)
Có-đường-đi(X, Z) ← Có-đường-đi(X, Y) ∧ Có-đường-đi(Y, Z)
```

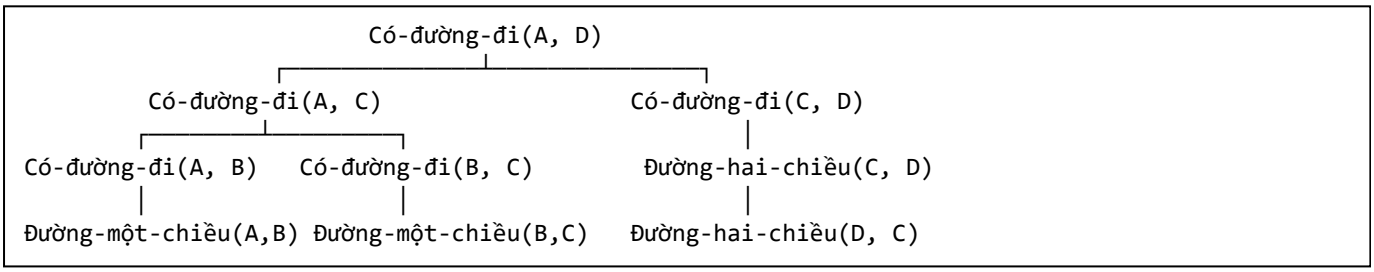
a) Hãy vẽ cây chứng minh cho khẳng định Có-đường-đi(A, D). b) Tìm điểm cố định của tiến trình suy diễn — viết chi tiết các bước mở rộng tập dữ kiện.

◇ Gợi ý / Khuôn

- **Cây chứng minh:** gốc là mệnh đề cần chứng minh. Mỗi nút được chọn một luật có **đầu (head)** khớp với nó, sau đó vẽ các mệnh đề trong **thân (body)** làm con. Đi xuống cho tới khi lá đều là **dữ kiện** có sẵn trong B0.
- **Điểm cố định:** bắt đầu từ B0 là tập dữ kiện. Ở mỗi bước, áp dụng **mọi luật** lên **toàn bộ tập của bước trước** và ghi lại các dữ kiện **mới**. Dừng khi một bước không thêm được dữ kiện nào; tập ở bước cuối có thêm dữ kiện chính là điểm cố định.
- Chi tiết hơn xem tab **D7 Datalog**.

▶ Đáp án

a) Cây chứng minh



Tên luật dùng ở từng nút:

- Gốc và nút (A,C) dùng luật bắc cầu `Có-đường-đi(X,Z) ← Có-đường-đi(X,Y) ∧ Có-đường-đi(Y,Z)`.
- Các nút (A,B) và (B,C) dùng luật `Có-đường-đi ← Đường-một-chiều`.
- Nút (C,D) dùng luật `Có-đường-đi ← Đường-hai-chiều`.
- Đường-hai-chiều(C,D) suy ra từ Đường-hai-chiều(D,C) nhờ luật đối xứng.

b) Điểm cố định

```
B0: {Đường-một-chiều(A, B), Đường-một-chiều(B, C), Đường-hai-chiều(D, C)}
B1: + {Đường-hai-chiều(C, D), Có-đường-đi(A, B), Có-đường-đi(B, C), Có-đường-đi(D, C)}
B2: + {Có-đường-đi(C, D), Có-đường-đi(A, C)}
B3: + {Có-đường-đi(B, D), Có-đường-đi(A, D), Có-đường-đi(C, C), Có-đường-đi(D, D)}
B4: + {}  (không có dữ kiện mới)
```

Điểm cố định thu được ở **Bước 3**. Tập điểm cố định gồm 3 dữ kiện ban đầu, 1 Đường-hai-chiều mới và 9 Có-đường-đi: AB, BC, DC, CD, AC, BD, AD, CC, DD.

Kiểm tra B4: từ A đi tới được B, C, D; từ B tới C, D; từ C tới C, D; từ D tới C, D. Mọi cặp bắc cầu đều đã có trong tập, nên B4 rỗng. Không suy ra được Có-đường-đi(A,A) vì không có đường nào quay về A.

D8 Viết truy vấn SPARQL (câu 2.5 điểm)

Câu 2.5 điểm: viết truy vấn SPARQL từ câu hỏi tiếng Việt.

③ 25 bài viết truy vấn (bài tập tuần, kết quả chạy thật)

Tóm tắt dữ liệu (để làm trên giấy):

- **Môn học:** Professor `hung` (dạy SemanticWeb, DataIntegration; tổ chức SemanticWeb), `lan` (dạy MachineLearning; tổ chức MachineLearning, DataIntegration); Assistant `minh` (dạy SemanticWebLab), `an` (dạy SemanticWebLab, MLLab). Course: SemanticWeb (3 tín chỉ), DataIntegration (2), MachineLearning (3); LaboratoryCourse: SemanticWebLab (1), MLLab (1). Homework hw1–hw3 ⊂ SemanticWeb, hw4 ⊂ SemanticWebLab, hw5 ⊂ MachineLearning. Mọi thứ có `:name`.
- **Phim:** Avatar 2009 (James Cameron; leading Sam, Zoe; supporting Michelle) · Titanic 1997 (Cameron; male-lead Leo, female-lead Kate; supporting Billy; có thêm tiêu đề `@vi`) · The Terminator 1984 (Cameron; leading Arnold; supporting Michael) · Inception 2010 (Nolan; leading Leo; supporting Michael) · Our Planet 2019 (không diễn viên) · Fantasia 1940 (AnimatedMovie, không diễn viên). Tiêu đề dùng `dc:title "…"@en`, tên người `foaf:name`.
- **Thư viện:** A Semantic Web Primer (Textbook, 2012, 288 tr., Antoniou + van Harmelen, MIT Press) · Knowledge Graphs (Textbook, 2021, 257 tr., Hogan, Springer; citedBy owlGuide) · The DL Handbook (Textbook, 2003, 574 tr., Baader, Cambridge; citedBy swPrimer, kgBook) · Semantic Web for the Working Ontologist (Book, 2011, *không có số trang*, Allemang + Hendler, Morgan Kaufmann) · Truyện Kiều (Novel, 1820, 254 tr., Nguyễn Du, *không có NXB*; tiêu đề @vi và @en).

■ Bộ 1 — Ontology môn học (Part 3 Ex3) · dữ liệu `ex3-lab-course`

Q1. Liệt kê tên giáo viên và tên môn họ dạy, sắp theo tên giáo viên.

▶ Lời giải — q01

```
PREFIX :     <http://example.org/course#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?teacherName ?courseName
WHERE {
  ?t :teaches ?c ;
     :name ?teacherName .
  ?c :name ?courseName .
}
ORDER BY ?teacherName ?courseName
```

?teacherName	?courseName
"An"	"Machine Learning Lab"
"An"	"Semantic Web Lab"
"Hung"	"Data Integration"
"Hung"	"Semantic Web"
"Lan"	"Machine Learning"
"Minh"	"Semantic Web Lab"

Q2. Các trợ giảng (Assistant) và môn thực hành họ dạy.

▶ Lời giải — q02

```
PREFIX :     <http://example.org/course#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?assistant ?lab
WHERE {
  ?a a :Assistant ; :name ?assistant ; :teaches ?l .
  ?l a :LaboratoryCourse ; :name ?lab .
}
ORDER BY ?assistant ?lab
```

?assistant	?lab
"An"	"Machine Learning Lab"
"An"	"Semantic Web Lab"
"Minh"	"Semantic Web Lab"

Q3. Môn học (kể cả môn thực hành) KHÔNG có bài tập về nhà nào.

▶ Lời giải — q03

```
PREFIX :     <http://example.org/course#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?courseName
WHERE {
  { ?c a :Course } UNION { ?c a :LaboratoryCourse }
  ?c :name ?courseName .
  FILTER NOT EXISTS { ?hw :isPartOf ?c }
}
ORDER BY ?courseName
```

?courseName
"Data Integration"
"Machine Learning Lab"

Q4. Số bài tập về nhà của mỗi môn (môn không có bài tập phải hiện 0), sắp giảm dần.

▶ Lời giải — q04

```
PREFIX :     <http://example.org/course#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?courseName (COUNT(?hw) AS ?n)
WHERE {
  ?c :credits ?cr ; :name ?courseName .
  OPTIONAL { ?hw :isPartOf ?c }
}
GROUP BY ?courseName
ORDER BY DESC(?n) ?courseName
```

?courseName	?n
"Semantic Web"	"3"
"Machine Learning"	"1"
"Semantic Web Lab"	"1"
"Data Integration"	"0"

?courseName	?n
"Machine Learning Lab"	"0"

Q5. Giáo viên vừa dạy vừa tổ chức CÙNG một môn.

▶ Lời giải — q05

```
PREFIX :     <http://example.org/course#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?teacherName ?courseName
WHERE {
  ?t :teaches ?c ; :organizes ?c ; :name ?teacherName .
  ?c :name ?courseName .
}
```

?teacherName	?courseName
"Hung"	"Semantic Web"
"Lan"	"Machine Learning"

Q6. Tất cả Teacher. SPARQL KHÔNG suy diễn: '?t a :Teacher' trả về rỗng vì dữ liệu chỉ ghi :Professor / :Assistant -> dùng property path rdfs:subClassOf* để tự đi lên cây lớp.

▶ Lời giải — q06

```
PREFIX :     <http://example.org/course#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?name ?type
WHERE {
  ?t rdf:type ?type ; :name ?name .
  ?type rdfs:subClassOf* :Teacher .
}
ORDER BY ?name
```

?name	?type
"An"	:Assistant
"Hung"	:Professor
"Lan"	:Professor
"Minh"	:Assistant

Q7. Tổng số tín chỉ mỗi giáo viên dạy; chỉ giữ người dạy >= 3 tín chỉ.

▶ Lời giải — q07

```
PREFIX :     <http://example.org/course#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?teacherName (SUM(?cr) AS ?total)
WHERE {
  ?t :teaches ?c ; :name ?teacherName .
  ?c :credits ?cr .
}
GROUP BY ?teacherName
HAVING (SUM(?cr) >= 3)
ORDER BY DESC(?total) ?teacherName
```

?teacherName	?total
"Hung"	"5"
"Lan"	"3"

Q8. ASK: có trợ giảng nào dạy môn KHÔNG phải môn thực hành không? (kiểm tra ràng buộc 'only')

▶ Lời giải — q08

```
PREFIX :     <http://example.org/course#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
ASK {
  ?a a :Assistant ; :teaches ?c .
  FILTER NOT EXISTS { ?c a :LaboratoryCourse }
}
```

Kết quả: **false**

Q9. CONSTRUCT: sinh các bộ ba nghịch đảo :isTaughtBy từ :teaches.

▶ Lời giải — q09

```
PREFIX :     <http://example.org/course#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
CONSTRUCT { ?c :isTaughtBy ?t }
WHERE     { ?t :teaches ?c }
```

Đồ thị kết quả:

```
:DataIntegration :isTaughtBy :hung .
:MLLab :isTaughtBy :an .
:MachineLearning :isTaughtBy :lan .
:SemanticWeb :isTaughtBy :hung .
:SemanticWebLab :isTaughtBy :an .
:SemanticWebLab :isTaughtBy :minh .
```

Q10. Mỗi môn kèm người tổ chức NẾU CÓ (OPTIONAL) — môn chưa có người tổ chức vẫn hiện.

▶ Lời giải — q10

```
PREFIX :     <http://example.org/course#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?courseName ?organizerName
WHERE {
  ?c :credits ?cr ; :name ?courseName .
  OPTIONAL { ?o :organizes ?c ; :name ?organizerName }
}
ORDER BY ?courseName ?organizerName
```

?courseName	?organizerName
"Data Integration"	"Lan"
"Machine Learning"	"Lan"
"Machine Learning Lab"	-
"Semantic Web"	"Hung"
"Semantic Web Lab"	-

■ Bộ 2 — Ontology phim (Part 3 Ex4) · dữ liệu `ex4-film`

Q11. Tên phim tiếng Anh và năm phát hành, sắp theo năm tăng dần.

▶ Lời giải — q11

```
PREFIX :      <http://example.org/film#>
PREFIX film:  <http://example.org/film/>
PREFIX people:<http://example.org/people/>
PREFIX dc:    <http://purl.org/dc/elements/1.1/>
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX rdfs:  <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:   <http://www.w3.org/2001/XMLSchema#>
SELECT ?title ?year
WHERE {
  ?m :releaseYear ?year ; dc:title ?title .
  FILTER (lang(?title) = "en")
}
ORDER BY ?year
```

?title	?year
"Fantasia"@en	"1940"^^xsd:gYear
"The Terminator"@en	"1984"^^xsd:gYear
"Titanic"@en	"1997"^^xsd:gYear
"Avatar"@en	"2009"^^xsd:gYear
"Inception"@en	"2010"^^xsd:gYear
"Our Planet"@en	"2019"^^xsd:gYear

Q12. Phim do 'James Cameron' đạo diễn, phát hành sau năm 1990.

▶ Lời giải — q12

```
PREFIX :      <http://example.org/film#>
PREFIX film:  <http://example.org/film/>
PREFIX people:<http://example.org/people/>
PREFIX dc:    <http://purl.org/dc/elements/1.1/>
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX rdfs:  <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:   <http://www.w3.org/2001/XMLSchema#>
SELECT ?title ?year
WHERE {
  ?m :director ?d ; dc:title ?title ; :releaseYear ?year .
  ?d foaf:name "James Cameron" .
  FILTER (lang(?title) = "en" && xsd:integer(str(?year)) > 1990)
}
ORDER BY ?year
```

?title	?year
"Titanic"@en	"1997"^^xsd:gYear
"Avatar"@en	"2009"^^xsd:gYear

Q13. Mọi diễn viên của mọi phim, dù dùng :leading-actor, :supporting-actor, :male-lead hay :female-lead. Không có suy diễn subPropertyOf -> tự đi theo rdfs:subPropertyOf*.

▶ Lời giải — q13

```
PREFIX :      <http://example.org/film#>
PREFIX film:  <http://example.org/film/>
PREFIX people:<http://example.org/people/>
PREFIX dc:    <http://purl.org/dc/elements/1.1/>
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX rdfs:  <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:   <http://www.w3.org/2001/XMLSchema#>
SELECT DISTINCT ?title ?actorName
WHERE {
  ?m ?p ?a ; dc:title ?title .
  ?p rdfs:subPropertyOf* :actor .
  ?a foaf:name ?actorName .
  FILTER (lang(?title) = "en")
}
ORDER BY ?title ?actorName
```

?title	?actorName
"Avatar"@en	"Michelle Rodriguez"
"Avatar"@en	"Sam Worthington"
"Avatar"@en	"Zoe Saldana"
"Inception"@en	"Leonardo DiCaprio"
"Inception"@en	"Michael Biehn"
"The Terminator"@en	"Arnold Schwarzenegger"
"The Terminator"@en	"Michael Biehn"
"Titanic"@en	"Billy Zane"
"Titanic"@en	"Kate Winslet"
"Titanic"@en	"Leonardo DiCaprio"

Q14. Phim không có diễn viên nào (ứng viên AnimalDocumentary / AnimatedMovie).

▶ Lời giải — q14

```
PREFIX :      <http://example.org/film#>
PREFIX film:  <http://example.org/film/>
PREFIX people:<http://example.org/people/>
PREFIX dc:    <http://purl.org/dc/elements/1.1/>
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX rdfs:  <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:   <http://www.w3.org/2001/XMLSchema#>
SELECT ?title
WHERE {
  ?m dc:title ?title .
  FILTER NOT EXISTS { ?m ?p ?a . ?p rdfs:subPropertyOf* :actor . }
}
ORDER BY ?title
```

?title
"Fantasia"@en
"Our Planet"@en

Q15. Diễn viên đóng từ 2 phim trở lên, kèm số phim.

▶ Lời giải — q15

```
PREFIX :      <http://example.org/film#>
PREFIX film:  <http://example.org/film/>
PREFIX people:<http://example.org/people/>
PREFIX dc:    <http://purl.org/dc/elements/1.1/>
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX rdfs:  <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:   <http://www.w3.org/2001/XMLSchema#>
SELECT ?name (COUNT(DISTINCT ?m) AS ?movies)
WHERE {
  ?m ?p ?a .
  ?p rdfs:subPropertyOf* :actor .
  ?a foaf:name ?name .
}
GROUP BY ?name
HAVING (COUNT(DISTINCT ?m) >= 2)
ORDER BY DESC(?movies) ?name
```

?name	?movies
"Leonardo DiCaprio"	"2"
"Michael Biehn"	"2"

Q16. Mỗi đạo diễn làm bao nhiêu phim và danh sách tên phim.

▶ Lời giải — q16

```
PREFIX :      <http://example.org/film#>
PREFIX film:  <http://example.org/film/>
PREFIX people:<http://example.org/people/>
PREFIX dc:    <http://purl.org/dc/elements/1.1/>
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX rdfs:  <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:   <http://www.w3.org/2001/XMLSchema#>
SELECT ?director (COUNT(?m) AS ?n) (GROUP_CONCAT(?title; separator=", ") AS ?titles)
WHERE {
  ?m :director ?d ; dc:title ?title .
  ?d foaf:name ?director .
  FILTER (lang(?title) = "en")
}
GROUP BY ?director
ORDER BY DESC(?n)
```

?director	?n	?titles
"James Cameron"	"3"	"Avatar, Titanic, The Terminator"
"Christopher Nolan"	"1"	"Inception"

Q17. Các cặp diễn viên cùng đóng một phim (mỗi cặp chỉ một lần).

▶ Lời giải — q17

```
PREFIX :      <http://example.org/film#>
PREFIX film:  <http://example.org/film/>
PREFIX people:<http://example.org/people/>
PREFIX dc:    <http://purl.org/dc/elements/1.1/>
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX rdfs:  <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:   <http://www.w3.org/2001/XMLSchema#>
SELECT DISTINCT ?n1 ?n2 ?title
WHERE {
  ?m ?p1 ?a1 ; ?p2 ?a2 ; dc:title ?title .
  ?p1 rdfs:subPropertyOf* :actor .
  ?p2 rdfs:subPropertyOf* :actor .
  ?a1 foaf:name ?n1 . ?a2 foaf:name ?n2 .
  FILTER (str(?n1) < str(?n2) && lang(?title) = "en")
}
ORDER BY ?title ?n1 ?n2
```

?n1	?n2	?title
"Michelle Rodriguez"	"Sam Worthington"	"Avatar"@en
"Michelle Rodriguez"	"Zoe Saldana"	"Avatar"@en
"Sam Worthington"	"Zoe Saldana"	"Avatar"@en
"Leonardo DiCaprio"	"Michael Biehn"	"Inception"@en
"Arnold Schwarzenegger"	"Michael Biehn"	"The Terminator"@en
"Billy Zane"	"Kate Winslet"	"Titanic"@en
"Billy Zane"	"Leonardo DiCaprio"	"Titanic"@en
"Kate Winslet"	"Leonardo DiCaprio"	"Titanic"@en

Q18. Tên phim tiếng Anh, kèm tên tiếng Việt nếu có (FILTER lang đặt TRONG OPTIONAL).

▶ Lời giải — q18

```
PREFIX :      <http://example.org/film#>
PREFIX film:  <http://example.org/film/>
PREFIX people:<http://example.org/people/>
PREFIX dc:    <http://purl.org/dc/elements/1.1/>
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX rdfs:  <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:   <http://www.w3.org/2001/XMLSchema#>
SELECT ?en ?vi
WHERE {
  ?m dc:title ?en .
  FILTER (lang(?en) = "en")
  OPTIONAL { ?m dc:title ?vi . FILTER (lang(?vi) = "vi") }
}
ORDER BY ?en
```

?en	?vi
"Avatar"@en	-
"Fantasia"@en	-
"Inception"@en	-
"Our Planet"@en	-
"The Terminator"@en	-
"Titanic"@en	"Titanic"@vi

■ Bộ 3 — Thư viện (Part 1 Ex7) · dữ liệu `library`

Q19. Mỗi cuốn sách (tiêu đề tiếng Anh) và danh sách tác giả.

▶ Lời giải — q19

```
PREFIX :     <http://example.org/library#>
PREFIX dc:   <http://purl.org/dc/terms/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?title (GROUP_CONCAT(?author; separator="; ") AS ?authors)
WHERE {
  ?b dc:title ?title ; :writtenBy ?a .
  ?a foaf:name ?author .
  FILTER langMatches(lang(?title), "en")
}
GROUP BY ?title
ORDER BY ?title
```

?title	?authors
"A Semantic Web Primer"@en	"Grigoris Antoniou; Frank van Harmelen"
"Knowledge Graphs"@en	"Aidan Hogan"
"Semantic Web for the Working Ontologist"@en	"Dean Allemang; James Hendler"
"The Description Logic Handbook"@en	"Franz Baader"
"The Tale of Kieu"@en	"Nguyễn Du"

Q20. Sách thiếu nhà xuất bản HOẶC thiếu số trang (mẫu OPTIONAL + !bound).

▶ Lời giải — q20

```
PREFIX :     <http://example.org/library#>
PREFIX dc:   <http://purl.org/dc/terms/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?title ?publisher ?pages
WHERE {
  ?b dc:title ?title .
  FILTER (lang(?title) = "en")
  OPTIONAL { ?b :publishedBy ?p . ?p foaf:name ?publisher }
  OPTIONAL { ?b :pages ?pages }
  FILTER (!bound(?publisher) || !bound(?pages))
}
ORDER BY ?title
```

?title	?publisher	?pages
"Semantic Web for the Working Ontologist"@en	"Morgan Kaufmann"	-
"The Tale of Kieu"@en	-	"254"

Q21. Mọi sách trích dẫn DL Handbook trực tiếp hoặc bắc cầu (:citedBy+).

▶ Lời giải — q21

```
PREFIX :     <http://example.org/library#>
PREFIX dc:   <http://purl.org/dc/terms/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?title
WHERE {
  :dlHandbook :citedBy+ ?b .
  ?b dc:title ?title .
}
ORDER BY ?title
```

?title
"A Semantic Web Primer"@en
"Knowledge Graphs"@en
"Semantic Web for the Working Ontologist"@en

Q22. Số trang trung bình, lớn nhất, nhỏ nhất của các sách có ghi số trang.

▶ Lời giải — q22

```
PREFIX :     <http://example.org/library#>
PREFIX dc:   <http://purl.org/dc/terms/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT (AVG(?p) AS ?avg) (MAX(?p) AS ?max) (MIN(?p) AS ?min) (COUNT(?b) AS ?n)
WHERE { ?b :pages ?p }
```

?avg	?max	?min	?n
"343.25"	"574"	"254"	"4"

Q23. Tác giả có tên chứa 'an' (không phân biệt hoa thường).

▶ Lời giải — q23

```
PREFIX :     <http://example.org/library#>
PREFIX dc:   <http://purl.org/dc/terms/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?name
WHERE {
  ?a a :Author ; foaf:name ?name .
  FILTER regex(?name, "an", "i")
}
ORDER BY ?name
```

?name
"Aidan Hogan"
"Dean Allemang"
"Frank van Harmelen"
"Franz Baader"
"Grigoris Antoniou"

Q24. Tất cả :Book (kể cả :Textbook, :Novel) qua rdf:type/rdfs:subClassOf*, sách mới nhất trước.

▶ Lời giải — q24

```
PREFIX :     <http://example.org/library#>
PREFIX dc:   <http://purl.org/dc/terms/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?title ?year
WHERE {
  ?b a/rdfs:subClassOf* :Book ; dc:title ?title ; :year ?year .
  FILTER (lang(?title) = "en")
}
ORDER BY DESC(?year)
```

?title	?year
"Knowledge Graphs"@en	"2021"^^xsd:gYear
"A Semantic Web Primer"@en	"2012"^^xsd:gYear
"Semantic Web for the Working Ontologist"@en	"2011"^^xsd:gYear
"The Description Logic Handbook"@en	"2003"^^xsd:gYear
"The Tale of Kieu"@en	"1820"^^xsd:gYear

Q25. VALUES + BIND: với 2 nhà xuất bản cho trước, liệt kê sách và 'tuổi' sách tính đến 2026.

▶ Lời giải — q25

```
PREFIX :     <http://example.org/library#>
PREFIX dc:   <http://purl.org/dc/terms/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>
SELECT ?publisher ?title ?age
WHERE {
  VALUES ?publisher { "Springer" "MIT Press" }
  ?p foaf:name ?publisher .
  ?b :publishedBy ?p ; dc:title ?title ; :year ?y .
  BIND (2026 - xsd:integer(str(?y)) AS ?age)
}
ORDER BY ?age
```

?publisher	?title	?age
"Springer"	"Knowledge Graphs"@en	"5"
"MIT Press"	"A Semantic Web Primer"@en	"14"

---

④ Đề thi cũ dạng này

■ 2023-3 · Câu 4 — SPARQL + đại số (máy bay)

> ***Dạng:*** *D8 + D9* · ***Nguồn:*** *2023-C4*

```
@prefix : <http://semweb.edu.vn/exam#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
:Caspar a :Seaplane; :crew 2; :flight 1900.
:Convair a :Seaplane; :crew 1; :flight 1953.
:Su27 a :JetFighter; :crew 1.
:Mig35 a :JetFighter; :speed 2400.
:Mi26 a :Helicopter; :speed 295; :crew 5; :flight 1983; rdfs:label "Mil Mi-26".
```

a) Viết 1 truy vấn SPARQL tìm các dòng thủy phi cơ (:Seaplane) và các dòng trực thăng (:Helicopter). Dữ liệu đầu ra phải chứa IRI của dòng máy bay, và nhãn (rdfs:label) của nó nếu có hoặc bỏ trống nếu không. b) Viết biểu thức đại số SPARQL tương ứng với yêu cầu truy vấn ở mục a, sau đó giải thích chi tiết từng bước thực hiện và các kết quả thu được.

> **◇ Gợi ý / Khuôn**
>
> - Cụm "**A và B**" ở đây nghĩa là lấy cả hai loại, tức là phép **hoặc** trên kiểu. Viết bằng `FILTER (?t = :A || ?t = :B)` hoặc bằng `{…} UNION {…}`.
> - Cụm "**nếu có, bỏ trống nếu không**" thì dùng `OPTIONAL`.
> - Dịch sang đại số: `OPTIONAL` thành **LeftJoin**. `FILTER` bọc **ngoài toàn bộ nhóm** chứa nó, kể cả khi nó được viết ở cuối nhóm.

> **▶ Đáp án**
>
> **a)**
>
> ```
> PREFIX : <http://semweb.edu.vn/exam#>
> PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
> SELECT *
> WHERE {
>   { ?p a ?t } OPTIONAL { ?p rdfs:label ?label }
>   FILTER (?t = :Seaplane || ?t = :Helicopter)
> }
> ```
>
> Viết cách khác cũng đúng: `SELECT ?p ?label WHERE { {?p a :Seaplane} UNION {?p a :Helicopter} OPTIONAL {?p rdfs:label ?label} }`.

b) Biểu thức đại số

```
Filter( (?t = :Seaplane || ?t = :Helicopter),
        LeftJoin( BGP(?p a ?t), BGP(?p rdfs:label ?label) ) )
```

Các bước thực hiện

Bước 1. BGP(?p a ?t) cho 5 lời giải:

?p	?t
:Caspar	:Seaplane
:Convair	:Seaplane
:Su27	:JetFighter
:Mig35	:JetFighter
:Mi26	:Helicopter

Bước 2. BGP(?p rdfs:label ?label) cho 1 lời giải:

?p	?label
:Mi26	"Mil Mi-26"

Bước 3. LeftJoin giữ **mọi** dòng của vế trái. Dòng nào có dòng tương thích ở vế phải (cùng giá trị ?p) thì được ghép thêm ?label:

?p	?t	?label
:Caspar	:Seaplane	
:Convair	:Seaplane	
:Su27	:JetFighter	
:Mig35	:JetFighter	
:Mi26	:Helicopter	"Mil Mi-26"

Bước 4. Filter chỉ giữ những dòng có ?t = :Seaplane hoặc ?t = :Helicopter, tức là loại 2 dòng JetFighter. Đây cũng là **kết quả truy vấn**:

?p	?t	?label
:Caspar	:Seaplane	
:Convair	:Seaplane	
:Mi26	:Helicopter	"Mil Mi-26"

■ 2022-1 · Câu 5 — SPARQL + đại số (mèo của Anna)

Dạng: *D8 + D9* · ***Nguồn:*** *2022-C5*

```
@prefix bio: <http://exam.hust.edu.vn/bio/> .
bio:Fido a bio:Dog; bio:offspring bio:Spot, bio:Ted.
bio:Milo a bio:Cat; bio:offspring bio:Kitty.
bio:Anna bio:own bio:Fido, bio:Milo.
```

a) Hãy viết 1 truy vấn SPARQL xuất danh sách những con mèo của Anna, bao gồm cả những con mèo mà Anna sở hữu và cả con của chúng. b) Viết biểu thức đại số SPARQL tương ứng với truy vấn ở mục a và giải thích chi tiết từng bước thực hiện.

◇ Gợi ý / Khuôn

Đề nói "**bao gồm cả** A **và cả** B" nghĩa là kết quả là hợp của hai nhóm, nên dùng `UNION`:

- nhánh 1: mèo Anna sở hữu;
- nhánh 2: con của mèo Anna sở hữu.

Dịch sang đại số: `Project( Union( BGP1, BGP2 ), {?pet} )`.

▶ Đáp án

a)

```
PREFIX bio: <http://exam.hust.edu.vn/bio/>
SELECT ?pet
WHERE {
  { bio:Anna bio:own ?pet . ?pet a bio:Cat . }
  UNION
  { bio:Anna bio:own ?x . ?x a bio:Cat . ?x bio:offspring ?pet . }
}
```

b) Biểu thức đại số:

```
Project( Union( BGP(bio:Anna bio:own ?pet . ?pet a bio:Cat .),
                BGP(bio:Anna bio:own ?x . ?x a bio:Cat . ?x bio:offspring ?pet .) ),
         {?pet} )
```

Bước 1. BGP(bio:Anna bio:own ?pet . ?pet a bio:Cat .). Anna sở hữu Fido và Milo, trong đó chỉ Milo là Cat:

?pet
bio:Milo

Bước 2. BGP(bio:Anna bio:own ?x . ?x a bio:Cat . ?x bio:offspring ?pet .):

?x	?pet
bio:Milo	bio:Kitty

Bước 3. Union({μ1: ?pet→bio:Milo}, {μ2: ?x→bio:Milo, ?pet→bio:Kitty}):

?x	?pet
	bio:Milo
bio:Milo	bio:Kitty

Bước 4. Project lên biến ?pet:

?pet
bio:Milo
bio:Kitty

Kết quả gồm 2 tài nguyên là bio:Milo và bio:Kitty. Đã chạy thử bằng rdflib, kết quả khớp.

■ 6390 · Câu 4 — SPARQL (ngôn ngữ lập trình) + đại số

Dạng: *D8 + D9* · ***Nguồn:*** *6390-C4*

```
@prefix : <http://semweb.edu.vn/exam#> .
:CPP a :PL; :creator :bjarne_stroustrup; :hasFeature :type, :function, :pointer, :class,
:template.
:C a :PL; :creator :dennis_ritchie; :hasFeature :type, :pointer, :function.
:Java a :PL; :creator :james_gosling; :hasFeature :type, :class.
:Python :hasFeature :function, :class .
:Lisp a :PL; :hasFeature :function.
:Ruby a :PL.
```

a) Viết truy vấn tìm các ngôn ngữ lập trình có tính năng (hasFeature) con trỏ (pointer) hoặc lớp (class). b) Tìm các ngôn ngữ lập trình (PL) cùng với người sáng lập nếu có dữ liệu, để trống cột người sáng lập nếu không có dữ liệu. c) Tìm và khẳng định (construct) các ngôn ngữ là :OOPL (NNLT HĐT) nếu có tính năng lớp. d) Viết biểu thức đại số cho mục a và giải thích chi tiết từng bước thực hiện biểu thức.

◇ Gợi ý / Khuôn

- (a) Chữ "**hoặc**" ứng với `UNION`, hoặc `FILTER(?f = :pointer || ?f = :class)`. Thêm `DISTINCT` vì C++ có cả hai tính năng, nếu không sẽ bị lặp 2 lần. Cụm "ngôn ngữ lập trình" nên ứng với `?pl a :PL`. Khi đó **Python bị loại** vì không có `a :PL`; nên ghi chú điều này.
- (b) Cụm "nếu có … để trống nếu không" ứng với `OPTIONAL`.
- (c) Cụm "khẳng định là …" ứng với `CONSTRUCT { ?pl a :OOPL } WHERE { … }`.

▶ Đáp án

a)

```
PREFIX : <http://semweb.edu.vn/exam#>
SELECT DISTINCT ?pl
WHERE {
  ?pl a :PL .
  { ?pl :hasFeature :pointer } UNION { ?pl :hasFeature :class }
}
```

?pl
:CPP
:C
:Java

Nếu bỏ điều kiện `?pl a :PL` thì kết quả có thêm :Python, vì Python có tính năng :class nhưng dữ liệu không khai báo nó là :PL.

b)

```
PREFIX : <http://semweb.edu.vn/exam#>
SELECT ?pl ?creator
WHERE {
  ?pl a :PL .
  OPTIONAL { ?pl :creator ?creator }
}
```

?pl	?creator
:CPP	:bjarne_stroustrup
:C	:dennis_ritchie
:Java	:james_gosling
:Lisp	
:Ruby	

c)

```
PREFIX : <http://semweb.edu.vn/exam#>
CONSTRUCT { ?pl a :OOPL }
WHERE { ?pl :hasFeature :class }
```

Kết quả là đồ thị gồm 3 triple: `:CPP a :OOPL . :Java a :OOPL . :Python a :OOPL .` Nếu thêm `?pl a :PL` vào WHERE thì Python bị loại.

d) Biểu thức đại số cho mục a:

```
Distinct( Project( Join( BGP(?pl a :PL),
                         Union( BGP(?pl :hasFeature :pointer),
                                BGP(?pl :hasFeature :class) ) ),
                  {?pl} ) )
```

Bước 1. BGP(?pl a :PL):

?pl
:CPP
:C
:Java
:Lisp
:Ruby

Bước 2. BGP(?pl :hasFeature :pointer):

?pl
:CPP
:C

Bước 3. BGP(?pl :hasFeature :class):

?pl
:CPP
:Java
:Python

Bước 4. Union(Bước 2, Bước 3) là hợp đa tập, giữ cả các dòng trùng nhau:

?pl
:CPP
:C
:CPP
:Java
:Python

Bước 5. Join(Bước 1, Bước 4) chỉ giữ các cặp μ tương thích, tức là cùng giá trị ?pl. :Python bị loại vì không có trong Bước 1:

?pl
:CPP
:C
:CPP
:Java

Bước 6. Project lên {?pl} không đổi bảng (bảng chỉ có một biến). Bước 7. Distinct bỏ dòng :CPP bị trùng. **Kết quả:**

?pl
:CPP
:C
:Java

D9 Đại số SPARQL: dịch query → bảng từng bước

Đi kèm câu SPARQL: dịch truy vấn sang biểu thức đại số và lập bảng kết quả từng bước.

③ Bài luyện

■ Luyện tập 1 — FILTER bên trong OPTIONAL và FILTER bên ngoài

Dạng: *D9 ·* ***Nguồn:*** *bài tự luyện (dữ liệu máy bay 2023-C4)*

```
PREFIX : <http://semweb.edu.vn/exam#>
SELECT ?x ?f
WHERE { ?x a :Seaplane . OPTIONAL { ?x :flight ?f FILTER(?f > 1950) } }
```

a) Viết biểu thức đại số và giải từng bước. b) Nếu chuyển FILTER ra ngoài OPTIONAL thì kết quả thay đổi thế nào?

◇ Gợi ý / Khuôn

FILTER nằm **trong** OPTIONAL trở thành điều kiện thứ 3 của LeftJoin. Dòng nào của vế trái không thỏa điều kiện thì **vẫn được giữ**, chỉ là biến ?f để trống.

▶ Đáp án

a) Biểu thức:

```
Project( LeftJoin( BGP(?x a :Seaplane), BGP(?x :flight ?f), (?f > 1950) ), {?x, ?f} )
```

Bước 1. BGP(?x a :Seaplane):

?x
:Caspar
:Convair

Bước 2. BGP(?x :flight ?f):

?x	?f
:Caspar	1900
:Convair	1953
:Mi26	1983

Bước 3. LeftJoin với điều kiện ?f > 1950:

- Caspar ghép được với 1900, nhưng 1900 > 1950 là sai, nên giữ Caspar và để trống ?f.
- Convair ghép với 1953, điều kiện đúng, nên giữ cả dòng.

?x	?f
:Caspar	
:Convair	1953

Bước 4. Project lên {?x, ?f} không đổi bảng. **Kết quả có 2 dòng**, đã kiểm bằng rdflib.

b) Khi chuyển FILTER ra ngoài, biểu thức là `Filter( ?f > 1950, LeftJoin( BGP1, BGP2 ) )`. LeftJoin cho 2 dòng (Caspar 1900, Convair 1953). Filter loại Caspar, nên kết quả **chỉ còn Convair 1953**.

Kết luận: vị trí của FILTER làm thay đổi kết quả.

■ Luyện tập 2 — Union + Join + Filter

> ***Dạng:*** *D9* · ***Nguồn:*** *bài tự luyện (dữ liệu máy bay 2023-C4)*

```
PREFIX : <http://semweb.edu.vn/exam#>
SELECT ?x ?c
WHERE { { ?x a :Seaplane } UNION { ?x a :JetFighter }
        ?x :crew ?c
        FILTER(?c >= 2) }
```

▶ Đáp án

Biểu thức:

```
Project( Filter( ?c >= 2,
                 Join( Union( BGP(?x a :Seaplane), BGP(?x a :JetFighter) ),
                       BGP(?x :crew ?c) ) ),
         {?x, ?c} )
```

Bước	Bảng
Union	?x = Caspar, Convair, Su27, Mig35
BGP(?x :crew ?c)	(Caspar,2), (Convair,1), (Su27,1), (Mi26,5)
Join	(Caspar,2), (Convair,1), (Su27,1). Mig35 không có crew; Mi26 không có trong Union
Filter ?c ≥ 2	(Caspar,2)

Kết quả có 1 dòng: :Caspar, 2, đã kiểm bằng rdflib.

FILTER được viết ở cuối nhóm nhưng vẫn **bọc toàn bộ nhóm**, tức là bọc cả Join chứ không chỉ bọc BGP crew.

■ Luyện tập 3 — GROUP BY + COUNT/SUM

> ***Dạng:*** *D9* · ***Nguồn:*** *bài tự luyện (dữ liệu máy bay 2023-C4)*

```
PREFIX : <http://semweb.edu.vn/exam#>
SELECT ?t (COUNT(?x) AS ?n) (SUM(?c) AS ?tong)
WHERE { ?x a ?t . ?x :crew ?c }
GROUP BY ?t
```

▶ Đáp án

Biểu thức:

```
Project( Extend( Extend( Group( (?t), BGP(?x a ?t . ?x :crew ?c) ),
                         ?n, COUNT(?x) ),
                 ?tong, SUM(?c) ),
         {?t, ?n, ?tong} )
```

Bước 1. BGP:

?x	?t	?c
:Caspar	:Seaplane	2
:Convair	:Seaplane	1
:Su27	:JetFighter	1
:Mi26	:Helicopter	5

Bước 2. Group theo ?t, được 3 nhóm:

- Seaplane gồm {Caspar, Convair};

- JetFighter gồm {Su27};
- Helicopter gồm {Mi26}.

Mig35 không xuất hiện vì không có :crew.

Bước 3. Tính hàm tổng hợp (Aggregation / Extend) cho mỗi nhóm. Bước 4. Project:

?t	?n	?tong
:Seaplane	2	3
:JetFighter	1	1
:Helicopter	1	5

Đã kiểm bằng rdflib.

④ Đề thi cũ dạng này

Câu đại số SPARQL trong đề cũ luôn là **phần (b)** của câu SPARQL. Lời giải đủ bảng từng bước nằm ở **[D8] ④, trang 54** (2023-3 Câu 4b, 2022-1 Câu 5b, 6390 Câu 4d).

D10 SHACL: khuôn hình, nút được kiểm, vi phạm

Đề bắt viết khuôn hình SHACL, chỉ ra nút nào được kiểm, nút nào vi phạm và sửa thế nào.

③ Bài luyện

■ Luyện tập 1 — Nhãn duy nhất

> ***Dạng:*** *D10* · ***Nguồn:*** *bài tự luyện (dữ liệu 2023-C4)*

Mỗi trực thăng (:Helicopter) phải có **đúng 1** nhãn rdfs:label kiểu chuỗi. Viết shape và cho biết kết quả kiểm tra.

▶ Đáp án

```
@prefix : <http://semweb.edu.vn/exam#> .
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
:HelicopterShape a sh:NodeShape ;
    sh:targetClass :Helicopter ;
    sh:property [ sh:path rdfs:label ;
                  sh:minCount 1 ; sh:maxCount 1 ;
                  sh:datatype xsd:string ] .
```

Shape chỉ kiểm tra một focus node là :Mi26. Mi26 có đúng 1 nhãn "Mil Mi-26", kiểu xsd:string, nên **đạt**. Kết quả: `sh:conforms true`.

■ Luyện tập 2 — Khoảng giá trị, chọn target theo thuộc tính

> ***Dạng:*** *D10* · ***Nguồn:*** *bài tự luyện (dữ liệu 2023-C4)*

Mọi máy bay **có khai báo** :crew thì số phi hành đoàn phải là số nguyên từ 1 đến 4. Nút nào được kiểm tra, nút nào vi phạm?

▶ Đáp án

```
:CrewShape a sh:NodeShape ;
    sh:targetSubjectsOf :crew ;
    sh:property [ sh:path :crew ; sh:datatype xsd:integer ;
                  sh:minInclusive 1 ; sh:maxInclusive 4 ] .
```

Focus node	:crew	Kết quả
:Caspar	2	đạt
:Convair	1	đạt
:Su27	1	đạt
:Mi26	5	**vi phạm** (maxInclusive 4)

- **Mig35 không được kiểm tra**, vì nó không đứng ở vị trí chủ ngữ của :crew.
- Sửa: hoặc đổi dữ liệu của Mi26 cho đúng thực tế, hoặc nới ràng buộc lên `sh:maxInclusive 10`.

■ Luyện tập 3 — sh:class và OWL allValuesFrom

> ***Dạng:*** *D10 + D6* · ***Nguồn:*** *bài tự luyện (dữ liệu 2022-C5)*

Dữ liệu: `bio:Milo a bio:Cat; bio:offspring bio:Kitty.` `bio:Fido a bio:Dog; bio:offspring bio:Spot, bio:Ted.` Ràng buộc: con của mèo phải là mèo. a) Viết bằng SHACL và cho biết kết quả. b) So sánh với tiên đề OWL `Cat`

$\sqsubseteq \forall$`offspring.Cat`.

▶ Đáp án

```
@prefix bio: <http://exam.hust.edu.vn/bio/> .
@prefix sh: <http://www.w3.org/ns/shacl#> .
bio:CatShape a sh:NodeShape ;
    sh:targetClass bio:Cat ;
    sh:property [ sh:path bio:offspring ; sh:class bio:Cat ] .
```

a) Shape kiểm tra một focus node là Milo. Giá trị :Kitty **không có** triple `a bio:Cat`, nên **vi phạm sh:class**. Fido không được kiểm tra vì là Dog. Sửa: thêm `bio:Kitty a bio:Cat .`

b) Với OWL, reasoner **suy ra** `bio:Kitty a bio:Cat` và không báo lỗi gì. Đây chính là khác biệt suy diễn (OWA) và kiểm tra (CWA).

■ Luyện tập 4 — Giá trị liệt kê (sh:in)

Dạng: *D10* · ***Nguồn:*** *bài tự luyện (nối với 2023-C2 "nhiên liệu")*

Dữ liệu: `:amphicar :nhienLieu :xang . :tesla :nhienLieu :dien . :xeX :nhienLieu :gas . :xeY :nhienLieu :xang, :dien .` Ràng buộc:

- giá trị nhiên liệu chỉ có thể là xăng, dầu hoặc điện;
- mỗi xe khai báo **tối đa 1** nhiên liệu.

▶ Đáp án

```
:FuelShape a sh:NodeShape ;
    sh:targetSubjectsOf :nhienLieu ;
    sh:property [ sh:path :nhienLieu ;
                  sh:in ( :xang :dau :dien ) ;
                  sh:maxCount 1 ] .
```

Focus node	Giá trị	Kết quả
:amphicar	:xang	đạt
:tesla	:dien	đạt
:xeX	:gas	**vi phạm sh:in** (:gas không có trong danh sách)
:xeY	:xang, :dien	**vi phạm sh:maxCount** (có 2 giá trị)

Sửa dữ liệu:

- đổi giá trị của :xeX thành `:xang` hoặc `:dau`;
- với :xeY, bỏ bớt 1 giá trị, hoặc bỏ `sh:maxCount` nếu xe hybrid được phép dùng 2 nhiên liệu.

Khác với OWL: nếu dùng `owl:oneOf`, :gas không bị báo lỗi. Do OWL không có giả thiết tên duy nhất, reasoner sẽ suy ra :gas phải trùng với một trong ba cá thể xăng, dầu, điện.

④ Đề thi cũ dạng này

■ 2023-3 · Câu 5 — SHACL

Dạng: *D10* · ***Nguồn:*** *2023-C5*

Tạo các khuôn hình SHACL biểu diễn các ràng buộc cho dữ liệu trong câu 4: a) Giá trị của thuộc tính năm cất cánh đầu tiên (:flight) của các dòng thủy phi cơ phải > 1900. b) Dữ liệu về các dòng máy bay chiến đấu phản lực (:JetFighter) phải có các thuộc tính :crew và :speed. *Gợi ý của đề: a) sh:minExclusive b) sh:minCount*

◇ Gợi ý / Khuôn

```
:TênShape a sh:NodeShape ;
    sh:targetClass :Lớp ;                 # chọn những nút cần kiểm tra
    sh:property [ sh:path :thuộc_tính ;   # thuộc tính bị ràng buộc
                  <ràng buộc> giá_trị ] .
```

Đề nói "> 1900" (lớn hơn hẳn) thì dùng `sh:minExclusive 1900`. Nếu đề nói "≥ 1900" thì dùng `sh:minInclusive`.

Đề nói "phải có" một thuộc tính thì dùng `sh:minCount 1`. Đề đòi 2 thuộc tính thì viết 2 khối `sh:property` riêng.

▶ Đáp án

```
@prefix : <http://semweb.edu.vn/exam#> .
@prefix sh: <http://www.w3.org/ns/shacl#> .
:SeaPlaneShape a sh:NodeShape ;
    sh:targetClass :Seaplane ;
    sh:property [
        sh:path :flight ;
        sh:minExclusive 1900
    ] .

:JetFighterShape a sh:NodeShape ;
    sh:targetClass :JetFighter ;
    sh:property [ sh:path :crew ;  sh:minCount 1 ] ;
    sh:property [ sh:path :speed ; sh:minCount 1 ] .
```

Kết quả kiểm tra (nên ghi thêm để ăn điểm trọn):

- `:SeaPlaneShape` kiểm tra 2 nút :Caspar và :Convair. **:Caspar vi phạm** vì 1900 không lớn hơn hẳn 1900. :Convair (1953) đạt.
- `:JetFighterShape` kiểm tra 2 nút :Su27 và :Mig35. **:Su27 vi phạm** vì thiếu :speed. **:Mig35 vi phạm** vì thiếu :crew.

■ 6390 · Câu 5 — SHACL

Dạng: *D10* · ***Nguồn:*** *6390-C5*

Kiểm duyệt dữ liệu với SHACL, sử dụng dữ liệu trong câu 4: a) Tạo hình trạng :PLFeatures đảm bảo mỗi NNLT (thuộc lớp :PL) được liệt kê tối thiểu 1 tính năng. b) Khi thực hiện kiểm tra theo :PLFeatures những nút nào sẽ được kiểm tra? Những nút nào vi phạm ràng buộc? Hãy đề xuất bổ sung dữ liệu đối với các nút vi phạm (nếu có) để đáp ứng yêu cầu.

◇ Gợi ý / Khuôn

- Cụm "thuộc lớp :PL" ứng với `sh:targetClass :PL`. Cụm "tối thiểu 1" ứng với `sh:minCount 1`.
- **Nút được kiểm tra (focus node)** là những nút có `rdf:type :PL`, hoặc `rdf:type :Sub` với `:Sub rdfs:subClassOf* :PL` có **trong data graph** (định nghĩa "SHACL instance"). SHACL **không suy kiểu** từ domain/range/OWL: :Python có tính năng nhưng không có `a :PL` (cũng không thuộc lớp con nào của :PL) nên **không được kiểm tra**.
- Muốn tìm nút vi phạm: đếm số giá trị :hasFeature của từng focus node, nút nào có 0 giá trị là vi phạm.

▶ Đáp án

a)

```
@prefix : <http://semweb.edu.vn/exam#> .
@prefix sh: <http://www.w3.org/ns/shacl#> .
:PLFeatures a sh:NodeShape ;
    sh:targetClass :PL ;
    sh:property [
        sh:path :hasFeature ;
        sh:minCount 1
    ] .
```

b)

Focus node	Số giá trị :hasFeature	Kết quả
:CPP	5	đạt
:C	3	đạt
:Java	2	đạt
:Lisp	1	đạt
:Ruby	0	**VI PHẠM** (minCount 1)

- **Nút được kiểm tra:** :CPP, :C, :Java, :Lisp, :Ruby, tức là mọi nút có `a :PL`.
- **:Python không được kiểm tra**, vì dữ liệu không có triple `:Python a :PL` (hay `a` một lớp con của :PL). SHACL kiểm tra theo đúng dữ liệu: chỉ đi theo `rdfs:subClassOf*` có sẵn trong data graph, không suy kiểu từ domain/range/OWL.
- **Nút vi phạm:** :Ruby, vì không có giá trị :hasFeature nào.
- **Đề xuất sửa:** bổ sung tính năng cho Ruby, ví dụ `:Ruby :hasFeature :class, :function .` (Ruby là ngôn ngữ hướng đối tượng). Có thể bổ sung thêm `:Python a :PL .` để Python cũng được kiểm tra; Python sẽ đạt vì có 2 tính năng.

Báo cáo kiểm tra (validation report) sẽ có dạng:

```
[ a sh:ValidationReport ; sh:conforms false ;
  sh:result [ a sh:ValidationResult ;
      sh:focusNode :Ruby ; sh:resultPath :hasFeature ;
      sh:sourceConstraintComponent sh:MinCountConstraintComponent ;
      sh:resultSeverity sh:Violation ] ] .
```